

PROYECTO DE COMPUTACIÓN GRÁFICA

Guía del código

Visualizador de Rectas

Explicación paso a paso de todo el código de la aplicación: qué hace cada archivo, cada función y cada bloque, y cómo funciona la lógica matemática que calcula y dibuja la recta. Escrita sin suponer conocimientos previos.

Archivos explicados: `main.py`, `pages/index.py`, `pages/icons.py`, `pages/Futuro.py`, `pages/puntos_colineales.py`, `test_app.py`, `requirements.txt`

Lenguaje: Python 3.10 o superior · Bibliotecas: CustomTkinter, CTKTable, Matplotlib, Pillow

Contenido

1. Qué hace la aplicación
2. Conceptos de programación que necesitas
 1. Programa, intérprete y archivo .py
 2. Variables y tipos de datos
 3. Funciones
 4. Decisiones y repeticiones
 5. Módulos e importaciones
 6. Errores y excepciones
 7. Texto con formato (f-strings)
 8. Programación por eventos
3. Conceptos de interfaz gráfica
4. Conceptos matemáticos
 1. Plano, incrementos y pendiente
 2. Ecuación y ángulo
 3. Rasterización y DDA
 4. Los 11 casos de trazo
5. Estructura del proyecto
6. main.py — la ventana principal
7. pages/index.py — el cambio de páginas
8. pages/icons.py — los iconos
9. pages/Futuro.py — la página informativa
10. pages/puntos_colineales.py — el corazón de la app
 1. Estado compartido y avisos de cambio de tema
 2. Funciones matemáticas
 3. Clasificación de los 11 casos
 4. Algoritmo de trayectoria (DDA)
 5. Dibujo del plano
 6. Construcción de la pantalla
 7. La función calculate()
 8. La función clear()
11. test_app.py — las pruebas automáticas
12. requirements.txt y cómo ejecutar
13. Recorrido completo: de «python main.py» a la recta dibujada
14. Glosario

1. Qué hace la aplicación

Es un programa de escritorio (una ventana de Windows, Linux o macOS) que recibe **dos puntos** del plano cartesiano, **A** y **B**, y con ellos:

- calcula la **pendiente** de la recta que los une, su **ordenada al origen**, su **ángulo** de inclinación y su **ecuación** $y = mx + b$;
- calcula cuánto cambia la X y cuánto cambia la Y al ir de A a B (**ΔX** y **ΔY**) y en qué sentido;
- identifica cuál de los **11 casos de trazo** es (un solo punto, horizontal, vertical, diagonal de 45°, acostada, parada o dibujada al revés);
- genera una lista de **puntos intermedios** («trayectoria») que van de A a B, paso a paso, y los muestra en una tabla;
- dibuja la recta, los puntos A y B y la trayectoria en un **plano cartesiano**;
- permite cambiar entre **modo oscuro** y **modo claro**, y navegar a una segunda página («Módulo Futuro») que anuncia funciones que aún no existen.

Así se organiza la ventana. Los textos en azul son los nombres de las variables del código que crean cada parte; los irás encontrando a lo largo de la guía.



Figura 1. Esquema de la ventana principal (no está a escala).

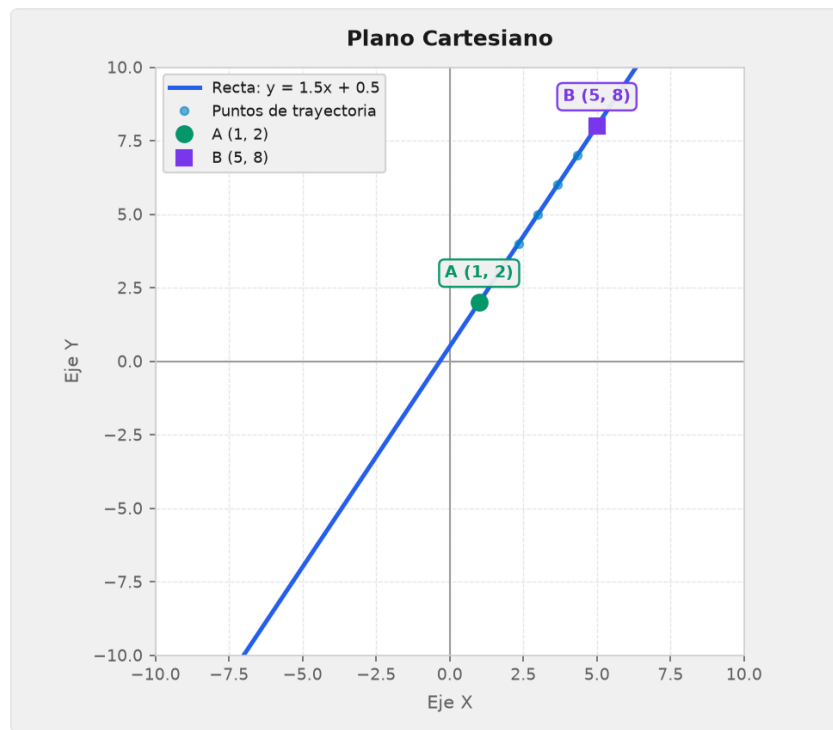


Figura 2. Gráfica real generada por la función `render_plot()` de la app con A(1, 2) y B(5, 8), en modo claro.

2. Conceptos de programación que necesitas

Esta sección explica, desde cero, las ideas de Python que aparecen en el código. Si ya las conoces, puedes saltar a la sección 5.

2.1 Programa, intérprete y archivo .py

Un **programa** es una lista de instrucciones que la computadora sigue en orden. En Python, esas instrucciones se escriben en archivos de texto con extensión `.py`. El **intérprete** de Python es el programa que lee ese archivo y ejecuta cada instrucción **de arriba hacia abajo**, una por una.

Cuando escribes en la terminal `python main.py`, le pides al intérprete que ejecute `main.py`. Las líneas que empiezan con `#` son **comentarios**: el intérprete las ignora, solo sirven para las personas. Los textos entre tres comillas `"""..."""` al inicio de un archivo o función son **docstrings**: también son descripciones para personas.

2.2 Variables y tipos de datos

Una **variable** es un nombre que guarda un valor. Se crea con `=`: `x1 = 5` guarda el número 5 en el nombre `x1`. Los valores tienen un **tipo**:

Tipo	Qué guarda	Ejemplo en el código
<code>int</code>	Número entero	<code>pasos = 6</code>
<code>float</code>	Número con decimales	<code>pendiente = 1.5</code>
<code>str</code>	Texto (cadena de caracteres), entre comillas	<code>"Ascendente"</code>
<code>bool</code>	Verdadero o falso: <code>True</code> / <code>False</code>	<code>"has_data": True</code>
<code>list</code>	Lista ordenada de valores, entre corchetes; se puede modificar	<code>[-10, 10]</code>
<code>tuple</code>	Lista que no se puede modificar, entre paréntesis	<code>(0, 1, 2), ("gray85", "gray25")</code>
<code>dict</code>	Diccionario: pares <i>clave</i> → <i>valor</i> , entre llaves	<code>{"X": x_line, "Y": y_line}</code>
<code>None</code>	«Nada», ausencia de valor	<code>{"table": None}</code>

Para leer un elemento de una lista o tupla se usa su **posición** empezando en 0: si `pt = (3, 5, 7)`, entonces `pt[0]` es 3, `pt[1]` es 5 y `pt[-1]` (el último) es 7. Para leer un diccionario se usa la **clave**: `current_data["pendiente"]`. El método `.get(clave, valor_por_defecto)` hace lo mismo pero, si la clave no existe, devuelve el valor por defecto en lugar de producir un error.

CONCEPTO · VARIABLE GLOBAL

Una variable creada fuera de cualquier función (al nivel del archivo) es **global**: todas las funciones del archivo pueden leerla. Si una función quiere *reemplazarla* por otro valor, debe declararlo con la palabra `global`. En este proyecto, `current_data`, `_pages` y `_nav_listeners` son globales. Por convención, un nombre que empieza con guion bajo (`_pages`) indica «uso interno de este archivo».

2.3 Funciones

Una **función** es un bloque de instrucciones con nombre que se puede ejecutar («llamar») muchas veces. Se define con `def`, recibe **parámetros** (datos de entrada) entre paréntesis y puede **devolver** un resultado con `return`. El cuerpo de la función es todo lo que está **sangrado** (con espacios a la izquierda) debajo del `def`.

Ejemplo real (pages/puntos_colineales.py)

```
def get_pendiente(x1, y1, x2, y2):  
    return (y2 - y1) / (x2 - x1)
```

```
m = get_pendiente(1, 2, 5, 8)    # m vale 1.5
```

- **Parámetros con valor por defecto:** `def setup_ejes(ax, figure, limit=10, is_dark=True)`. Si al llamarla no das `limit`, vale 10.
- **Argumentos con nombre:** `get_icon("logo", size=24)` indica explícitamente que 24 es el valor de `size`.
- **Funciones dentro de funciones:** en Python puedes definir una función dentro de otra. La función interior puede **leer las variables** de la exterior, incluso después de que la exterior terminó. A esto se le llama **cierre** (*closure*). Así funcionan `calculate()` y `clear()`: están dentro de `create_puntos_colineales_page()` y usan sus cajas de texto y etiquetas.
- **lambda:** una función de una sola línea y sin nombre. `command=lambda: show_page("futuro")` significa «cuando se presione el botón, ejecuta `show_page("futuro")`». Se usa `lambda` porque `command` necesita una función para llamar después, no el resultado de llamarla ahora.
- **Funciones como valores:** una función se puede guardar en una lista y llamarse después. Esto es un **callback** («función a la que se llama de vuelta»). Se usa en `_nav_listeners` y `_plot_callbacks`.

2.4 Decisiones y repeticiones

`if / elif / else` permite tomar decisiones:

```
if m > 0:  
    return "Ascendente"      # se ejecuta si m es mayor que 0  
elif m < 0:  
    return "Descendente"    # si no, pero m es menor que 0  
return "Horizontal"        # en cualquier otro caso (m == 0)
```

Operadores de comparación: `==` (igual), `!=` (distinto), `>`, `<`, `>=`, `<=`. Operadores lógicos: `and`, `or`, `not`. Observa que `=` **asigna** y `==` **compara**.

for repite un bloque para cada elemento de una colección. `range(n)` produce los números 0, 1, ..., n-1:

```
for i in range(3):    # i toma los valores 0, 1, 2
    print(i)
```

Listas por comprensión: `[pt[1] for pt in table_points]` crea una lista nueva tomando el elemento 1 de cada tupla de `table_points`. Es una forma corta de escribir un `for` que va agregando a una lista.

Desempaquetar: `for paso, px, py in table_points:` toma cada tupla de 3 elementos y guarda cada uno en su propia variable.

2.5 Módulos e importaciones

Cada archivo `.py` es un **módulo**. Una carpeta con módulos (como `pages/`) es un **paquete**. Para usar código de otro módulo se importa:

- `import math` — carga el módulo completo; se usa como `math.atan(...)`.
- `import customtkinter as ctk` — igual, pero con un apodo corto: `ctk.CTkButton(...)`.
- `from pages.icons import get_icon` — trae solo el nombre `get_icon` desde el archivo `pages/icons.py`.

CONCEPTO · IMPORTAR EJECUTA EL ARCHIVO

La primera vez que un módulo se importa, Python ejecuta **todo su código de nivel superior** (lo que no está dentro de una función). Por eso `main.py` protege la última línea con `if __name__ == "__main__":`. La variable especial `__name__` vale `"__main__"` solo cuando el archivo se ejecuta directamente (`python main.py`) y no cuando otro archivo lo importa.

2.6 Errores y excepciones

Cuando algo sale mal (por ejemplo, convertir el texto `"hola"` a número), Python **lanza una excepción**, y si nadie la atrapa, el programa se detiene. `try / except` permite atraparla y reaccionar:

```
try:
    x1 = float("hola")          # esto lanza ValueError
except ValueError:
    print("No es un número")    # en vez de detenerse, se ejecuta esto
```

`ValueError` es el tipo de error que produce `float()` cuando recibe texto no numérico. `except Exception:` atrapa casi cualquier error.

2.7 Texto con formato (f-strings)

Un texto que empieza con `f` puede incluir valores entre llaves: `f"A ({x1}, {y1})"` produce `"A (1, 2)"`. Después de dos puntos se indica el formato:

Formato	Significado	Ejemplo	Resultado
:g	Formato «general»: quita ceros y puntos innecesarios	<code>f"{5.0:g}", f"{1.6667:g}"</code>	5, 1.6667
:.1f	Exactamente 1 decimal	<code>f"{56.3099:.1f}"</code>	56.3
:+g	Formato general, siempre con signo	<code>f"{4:+g}", f"{-3:+g}"</code>	+4, -3

2.8 Programación por eventos

Un programa con ventana no se ejecuta de arriba abajo y termina. Primero **construye** la ventana y después entra en un **ciclo de eventos** (`root.mainloop()`): un bucle infinito que espera a que el usuario haga algo (clic, tecla, mover la ventana). Cuando ocurre un **evento**, llama a la función asociada (el `command` del botón) y vuelve a esperar. El ciclo termina cuando se cierra la ventana.

CONCEPTO · PATRÓN «OBSERVADOR»

Algunas partes del programa necesitan *enterarse* de que algo cambió (por ejemplo, la gráfica necesita saber que cambió el tema). En lugar de que el que cambia conozca a todos los interesados, se usa una **lista de callbacks**: cada interesado se «suscribe» agregando su función a la lista, y cuando ocurre el cambio se recorre la lista llamando a cada una. En este código lo hacen `register_nav_listener` / `show_page` y `register_plot_updater` / `notify_theme_to_plot`.

3. Conceptos de interfaz gráfica

3.1 Las bibliotecas

Biblioteca	Para qué se usa
Tkinter	Viene incluida con Python. Crea ventanas, botones, cajas de texto. Es la base de todo lo demás.
CustomTkinter (ctk)	Capa encima de Tkinter con aspecto moderno: esquinas redondeadas, modo oscuro/claro automático. Sus componentes empiezan con Ctk .
CTkTable	Componente de tabla con el estilo de CustomTkinter.
Matplotlib	Biblioteca de gráficas. Dibuja el plano cartesiano, la recta y los puntos.
Pillow (PIL)	Manipulación de imágenes. Se usa para dibujar los iconos.
math	Funciones matemáticas de Python (arcotangente, conversión a grados).

3.2 Widgets y jerarquía

Cada elemento visible es un **widget** (componente). Todo widget se crea **dentro de otro**, su «padre» (el primer argumento), formando un árbol: la ventana contiene marcos, los marcos contienen botones, etc.

Widget	Qué es
<code>ctk.CTk()</code>	La ventana principal. Solo hay una.
<code>CTkFrame</code>	Un marco o «caja» rectangular que agrupa otros widgets. Puede tener fondo, borde y esquinas redondeadas.
<code>CTkLabel</code>	Una etiqueta: muestra texto y/o una imagen. El usuario no la edita.
<code>CTkButton</code>	Un botón. Su parámetro <code>command</code> es la función que se ejecuta al hacer clic.
<code>CTkEntry</code>	Una caja de texto donde el usuario escribe. <code>.get()</code> lee su contenido, <code>.insert(0, "1")</code> escribe al inicio, <code>.delete(0, "end")</code> la vacía.
<code>CTkSwitch</code>	Un interruptor de encendido/apagado. <code>.get()</code> devuelve 1 (encendido) o 0 (apagado).
<code>CTkScrollableFrame</code>	Un marco con barra de desplazamiento.
<code>CTkImage</code>	Una imagen con dos versiones: una para modo claro y otra para modo oscuro.
<code>CTkFont</code>	Una fuente tipográfica: tamaño (<code>size</code>) y grosor (<code>weight="bold"</code> = negritas).

Para cambiar una propiedad de un widget después de creado se usa `.configure(...)`, por ejemplo `val_m_num.configure(text="1.5")`; para leerla, `.cget("text")`.

3.3 Cómo se acomodan los widgets: pack, grid y place

Crear un widget no lo muestra; hay que decir **dónde** va con un «gestor de geometría». Dentro de un mismo padre se debe usar un solo tipo de gestor.

- `pack()` — apila los widgets uno tras otro (de arriba abajo por defecto, o de izquierda a derecha con `side="left"`). `fill="x"` estira el widget a lo ancho, `fill="both"` a lo ancho y alto, y `expand=True` le da el espacio sobrante.
- `grid(row=, column=)` — acomoda en una cuadrícula de filas y columnas. `sticky="nsew"` (norte, sur, este, oeste) estira el widget para llenar toda su celda. `columnspan=4` hace que ocupe 4 columnas.
- `place(relx=0.5, rely=0.5, anchor="center")` — coloca en una posición relativa: 0.5 es la mitad, así que centra el widget.

Márgenes: `padx` y `pady` son el espacio vacío alrededor, en píxeles. Con una tupla, `pady=(12, 4)` significa 12 arriba y 4 abajo.

Peso (weight): `root.columnconfigure(1, weight=1)` indica que la columna 1 recibe el espacio extra cuando la ventana crece. Las columnas con peso 0 conservan su tamaño.

Propagación: normalmente un marco se encoge o crece para ajustarse a su contenido.

`grid_propagate(False)` o `pack_propagate(False)` desactivan eso para que el marco respete el ancho fijo que se le dio (por ejemplo, la barra lateral de 250 px).

3.4 Colores

Los colores se escriben como **hexadecimales** `#RRGGBB`: dos dígitos (de 00 a FF, es decir, de 0 a 255) para rojo, verde y azul. `#FFFFFF` es blanco, `#000000` negro, `#3B82F6` un azul. También se usan nombres como `"gray85"` (gris al 85 % de claridad) o `"transparent"`.

En CustomTkinter, un color puede darse como **tupla (claro, oscuro)**: `border_color=("gray85", "gray25")` usa gris claro en modo claro y gris oscuro en modo oscuro. CustomTkinter cambia entre ambos automáticamente al cambiar el tema.

RGBA agrega un cuarto número, la **opacidad** (alfa): 255 es opaco, 0 es totalmente transparente.

3.5 Matplotlib dentro de la ventana

- `Figure` — el «lienzo» completo de la gráfica.
- `figure.add_subplot(111)` — crea dentro del lienzo unos **ejes** (`ax`): el área con eje X y eje Y donde se dibuja. «111» significa «cuadrícula de 1×1, gráfica número 1».
- `FigureCanvasTkAgg(figure, master=...)` — un adaptador que convierte la figura de Matplotlib en un widget de Tkinter para poder meterla en la ventana.
- `canvas.draw()` — vuelve a pintar la figura en pantalla después de cambiarla.

4. Conceptos matemáticos

4.1 Plano cartesiano y puntos

El **plano cartesiano** son dos rectas numéricas perpendiculares: el **eje X** (horizontal) y el **eje Y** (vertical), que se cruzan en el **origen** (0, 0). Un **punto** se escribe (x, y): x indica cuánto se mueve a la derecha (o a la izquierda si es negativo) y y cuánto sube (o baja si es negativo). En la app, **A = (X₁, Y₁)** es el punto de origen y **B = (X₂, Y₂)** el de destino.

4.2 Incrementos ΔX y ΔY

La letra griega Δ («delta») significa «cambio» o «diferencia»:

$$\Delta X = X_2 - X_1 \quad \Delta Y = Y_2 - Y_1$$

El **signo** dice el sentido: ΔX positivo significa que B está a la derecha de A («Izquierda a Derecha»); negativo, a la izquierda. ΔY positivo significa que B está más arriba («Abajo a Arriba»). El **valor absoluto** |ΔX| (el número sin signo) es la **magnitud**: cuántas unidades se recorren.

4.3 Pendiente

La **pendiente** *m* mide qué tan inclinada está la recta: cuántas unidades sube (o baja) Y por cada unidad que avanza X.

$$m = \Delta Y / \Delta X = (Y_2 - Y_1) / (X_2 - X_1)$$

- $m > 0$: la recta sube de izquierda a derecha (**ascendente**).
- $m < 0$: la recta baja (**descendente**).
- $m = 0$: la recta es **horizontal**.
- Si $X_1 = X_2$, entonces $\Delta X = 0$ y la división es **entre cero**, que no está definida: la pendiente es **indefinida**. La recta es **vertical**, forma un ángulo de 90° con el eje X y su ecuación es $x = x_1$. La app la calcula y dibuja como un caso especial.
- Si además $Y_1 = Y_2$, A y B son **el mismo punto**: no hay recta, solo un punto.

4.4 Ordenada al origen y ecuación de la recta

Toda recta no vertical se puede escribir como

$$y = m \cdot x + b$$

donde *b* es la **ordenada al origen**: el valor de y cuando $x = 0$, es decir, donde la recta cruza el eje Y. Como A está sobre la recta, se cumple $Y_1 = m \cdot X_1 + b$; despejando:

$$b = Y_1 - m \cdot X_1$$

4.5 Ángulo de inclinación

El ángulo θ que forma la recta con el eje X cumple $\tan(\theta) = m$. Para obtener θ se usa la función inversa, la **arcotangente**: $\theta = \arctan(m)$. En Python, `math.atan(m)` devuelve el ángulo en **radianes** (otra unidad para medir ángulos: una vuelta completa son 2π radianes = 360°), y `math.degrees(...)` lo convierte a **grados**.

EJEMPLO COMPLETO · A(1, 2) Y B(5, 8)

$$\Delta X = 5 - 1 = 4 \cdot \Delta Y = 8 - 2 = 6$$

$$m = 6 / 4 = 1.5 \text{ (ascendente)}$$

$$b = 2 - 1.5 \times 1 = 0.5$$

$$\text{Ecuación: } y = 1.5x + 0.5$$

$$\theta = \arctan(1.5) = 0.9828 \text{ rad} = 56.3^\circ$$

$$\text{Comprobación con B: } 1.5 \times 5 + 0.5 = 8 \checkmark$$

4.6 Rasterización y el algoritmo DDA

Una pantalla está hecha de **píxeles**: una cuadrícula de puntos. Para dibujar una recta hay que decidir qué puntos intermedios pintar entre A y B; a eso se le llama **rasterizar**. Un método clásico es el **DDA** (*Digital Differential Analyzer*, analizador diferencial digital):

1. Se calcula el número de **pasos**: el mayor entre $|\Delta X|$ y $|\Delta Y|$. Así, en el eje que más cambia se avanza exactamente 1 unidad por paso.
2. Se calcula cuánto avanzar en cada eje por paso: `step_x =  $\Delta X$  / pasos` y `step_y =  $\Delta Y$  / pasos`.
3. Se parte de A y en cada paso i se calcula el punto $(X_1 + i \cdot \text{step}_x, Y_1 + i \cdot \text{step}_y)$, desde $i = 0$ (que es A) hasta $i = \text{pasos}$ (que es B).

La función `generar_puntos_trayectoria()` implementa exactamente esto (sección 10.4).

4.7 Los 11 casos de trazo

Según los signos y tamaños de ΔX y ΔY , toda recta de A a B cae en uno de estos casos. Distinguirlos es importante en graficación porque cada uno se rasteriza de forma distinta: en unos se avanza 1 en X por paso y en otros 1 en Y.

Caso	Condición	Ejemplo A → B	Qué pasa en cada paso
Un solo punto	$\Delta X = 0$ y $\Delta Y = 0$	$(2, 3) \rightarrow (2, 3)$	No hay recorrido
Horizontal hacia la derecha	$\Delta Y = 0, \Delta X > 0$	$(1, 2) \rightarrow (6, 2)$	X sube 1, Y igual
Horizontal hacia la izquierda	$\Delta Y = 0, \Delta X < 0$	$(6, 2) \rightarrow (1, 2)$	X baja 1, Y igual
Vertical hacia arriba	$\Delta X = 0, \Delta Y > 0$	$(3, 1) \rightarrow (3, 7)$	Y sube 1, X igual
Vertical hacia abajo	$\Delta X = 0, \Delta Y < 0$	$(3, 7) \rightarrow (3, 1)$	Y baja 1, X igual
Diagonal exacta (45°)	$ \Delta X = \Delta Y , \Delta X > 0$	$(0, 0) \rightarrow (5, 5)$	X y Y cambian 1
Acostada, subiendo hacia la derecha	$ \Delta Y < \Delta X , \Delta X > 0, \Delta Y > 0$	$(0, 0) \rightarrow (8, 3)$	X sube 1, Y sube menos de 1
Acostada, bajando hacia la derecha	$ \Delta Y < \Delta X , \Delta X > 0, \Delta Y < 0$	$(0, 5) \rightarrow (8, 2)$	X sube 1, Y baja menos de 1
Parada, subiendo hacia la derecha	$ \Delta Y > \Delta X , \Delta X > 0, \Delta Y > 0$	$(0, 0) \rightarrow (3, 8)$	Y sube 1, X sube menos de 1
Parada, bajando hacia la derecha	$ \Delta Y > \Delta X , \Delta X > 0, \Delta Y < 0$	$(0, 8) \rightarrow (3, 0)$	Y baja 1, X sube menos de 1
Dibujada al revés (de derecha a izquierda)	$\Delta X < 0$ y $\Delta Y \neq 0$	$(5, 8) \rightarrow (1, 2)$	X retrocede; equivale a otro caso recorrido de B hacia A

CONCEPTO · ACOSTADA Y PARADA

Una recta **acostada** está más cerca de la horizontal (inclinación menor de 45°, $|m| < 1$): en cada paso avanza 1 en X y un poco en Y. Una recta **parada** está más cerca de la vertical (inclinación mayor de 45°, $|m| > 1$): en cada paso avanza 1 en Y y un poco en X. La diagonal exacta ($|m| = 1$) es la frontera entre ambas.

5. Estructura del proyecto

.	
├─ main.py	← punto de entrada: crea la ventana y la barra lateral
├─ test_app.py	← pruebas automáticas
├─ requirements.txt	← lista de bibliotecas necesarias
├─ README.md	← descripción e instrucciones
├─ pages/	← paquete con las páginas y utilidades
│ └─ index.py	← administra qué página se muestra
│ └─ puntos_colineales.py	← página principal: matemáticas, tabla y gráfica
│ └─ Futuro.py	← página «Módulo en Desarrollo»
│ └─ icons.py	← dibuja los iconos

Quién usa a quién

Archivo	Importa de	Lo que toma
main.py	pages/index.py pages/puntos_colineales.py pages/icons.py	init_pages , show_page , register_nav_listener notify_theme_to_plot get_icon
pages/index.py	pages/Futuro.py pages/puntos_colineales.py	future_page create_puntos_colineales_page
pages/puntos_colineales.py	pages/icons.py + bibliotecas	get_icon
pages/Futuro.py	pages/icons.py	get_icon
test_app.py	pages/puntos_colineales.py pages/index.py	funciones matemáticas y de la página funciones de navegación

La idea general: `main.py` arma el «esqueleto» (ventana + barra lateral + contenedor vacío) y le pide a `index.py` que llene el contenedor con las páginas. Cada página es una función que construye un marco con todo su contenido y lo devuelve. `icons.py` es una utilidad que cualquiera puede usar para obtener iconos.

6. main.py — la ventana principal

Es el archivo que se ejecuta para abrir la app. Se lee de arriba abajo y construye la ventana en cuatro partes numeradas en sus comentarios.

6.1 Importaciones

main.py · líneas 1-9

```
1 """
2 Ventana Principal - Computación Gráfica
3 Construida con CustomTkinter, iconos vectoriales nativos (CTkImage) y switch interactivo.
4 """
5 import customtkinter as ctk
6
7 from pages.index import init_pages, show_page, register_nav_listener
8 from pages.puntos_colineales import notify_theme_to_plot
9 from pages.icons import get_icon
```

Líneas 1-4: docstring que describe el archivo. Línea 5: importa CustomTkinter con el apodo `ctk`. Líneas 7-9: traen funciones de los otros archivos del proyecto (ver tabla de la sección 5). Al importar `pages.index`, Python ejecuta ese archivo, que a su vez importa `Futuro.py` y `puntos_colineales.py`; así, al llegar a la línea 10, todas las funciones ya existen, aunque todavía no se ha creado ningún widget.

6.2 Tema y ventana

main.py · líneas 11-22

```
11 # 1. Configuración de tema visual
12 ctk.set_appearance_mode("Dark")
13 ctk.set_default_color_theme("blue")
14
15 # 2. Ventana Principal
16 root = ctk.CTk()
17 root.title("Computación Gráfica - Visualizador de Rectas")
18 root.geometry("1340x840")
19 root.minsize(1100, 720)
20
21 root.columnconfigure(1, weight=1)
22 root.rowconfigure(0, weight=1)
```

- **Línea 12:** la app inicia en modo oscuro.
- **Línea 13:** usa la paleta de colores azul de CustomTkinter para botones e interruptores.
- **Línea 16:** crea la ventana y la guarda en `root` («raíz»: el widget del que cuelgan todos los demás).
- **Líneas 17-19:** título de la ventana, tamaño inicial de 1340 × 840 píxeles y tamaño mínimo de 1100 × 720 (el usuario no puede achicarla más).
- **Líneas 21-22:** la ventana tendrá una cuadrícula: columna 0 para la barra lateral y columna 1 para el contenido. Con `weight=1`, la columna 1 y la fila 0 absorben todo el espacio extra al agrandar la ventana,

mientras la barra lateral conserva su ancho.

6.3 Barra lateral y encabezado

main.py · líneas 24-54

```
24 # 3. Barra Lateral (Sidebar)
25 sidebar = ctk.CTkFrame(root, width=250, corner_radius=0)
26 sidebar.grid(row=0, column=0, sticky="nsew")
27 sidebar.grid_propagate(False)
28 sidebar.grid_rowconfigure(4, weight=1)
29
30 # Encabezado con icono vectorial
31 header_box = ctk.CTkFrame(sidebar, fg_color="transparent")
32 header_box.pack(fill="x", padx=18, pady=(24, 4), anchor="w")
33
34 logo_icon = get_icon("logo", size=24, light_color="#2563EB", dark_color="#3B82F6")
35 logo_label = ctk.CTkLabel(
36     header_box,
37     text="  GRAFICACIÓN",
38     image=logo_icon,
39     compound="left",
40     font=ctk.CTkFont(size=17, weight="bold")
41 )
42 logo_label.pack(anchor="w")
43
44 logo_sub = ctk.CTkLabel(
45     sidebar,
46     text="Laboratorio de Algoritmos",
47     font=ctk.CTkFont(size=11),
48     text_color="gray"
49 )
50 logo_sub.pack(padx=22, pady=(2, 20), anchor="w")
51
52 # Separador
53 sep = ctk.CTkFrame(sidebar, height=1, fg_color=("gray80", "gray25"))
54 sep.pack(fill="x", padx=16, pady=(0, 16))
```

- **Línea 25:** un marco de 250 px de ancho sin esquinas redondeadas. **Línea 26:** se coloca en la celda (fila 0, columna 0) y con `sticky="nsew"` se estira hacia arriba y abajo. **Línea 27:** `grid_propagate(False)` hace que respete los 250 px aunque su contenido sea más angosto.
- **Línea 28:** da peso a la fila 4 de la cuadrícula de la barra lateral. *Nota:* como los hijos de la barra lateral se colocan con `pack` y no con `grid`, esta línea no tiene efecto visible.
- **Líneas 31-32:** un marco transparente para agrupar el logotipo.
- **Línea 34:** pide a `icons.py` el icono «logo» de 24 px, azul oscuro para modo claro y azul brillante para modo oscuro.
- **Líneas 35-42:** una etiqueta con el texto «GRAFICACIÓN» y el icono. `compound="left"` pone la imagen a la izquierda del texto. Los dos espacios al inicio del texto separan el texto del icono.
- **Líneas 44-50:** el subtítulo gris «Laboratorio de Algoritmos».
- **Líneas 53-54:** una línea separadora: un marco de 1 px de alto con color gris.

6.4 Botones de navegación

main.py · líneas 56-87

```
56 # Menú de Navegación con Iconos
57 nav_buttons = {}
58
59 icon_puntos = get_icon("trending-up", size=18, light_color="#1E293B", dark_color="#F8FAFC")
60 btn_puntos = ctk.CTkButton(
61     sidebar,
62     text=" Puntos Colineales",
63     image=icon_puntos,
64     compound="left",
65     height=40,
66     corner_radius=8,
67     anchor="w",
68     font=ctk.CTkFont(size=13, weight="bold"),
69     command=lambda: show_page("puntos_colineales")
70 )
71 btn_puntos.pack(fill="x", padx=14, pady=4)
72 nav_buttons["puntos_colineales"] = btn_puntos
73
74 icon_futuro = get_icon("sparkles", size=18, light_color="#1E293B", dark_color="#F8FAFC")
75 btn_futuro = ctk.CTkButton(
76     sidebar,
77     text=" Módulo Futuro",
78     image=icon_futuro,
79     compound="left",
80     height=40,
81     corner_radius=8,
82     anchor="w",
83     font=ctk.CTkFont(size=13),
84     command=lambda: show_page("futuro")
85 )
86 btn_futuro.pack(fill="x", padx=14, pady=4)
87 nav_buttons["futuro"] = btn_futuro
```

- **Línea 57:** un diccionario vacío que guardará los botones por nombre de página:
`{"puntos_colineales": btn_puntos, "futuro": btn_futuro}`. Sirve para después pintar de azul el botón de la página activa.
- **Líneas 59-71:** el botón «Puntos Colineales». `anchor="w"` alinea el texto a la izquierda (`w` = *west*, oeste). Su `command` es `lambda: show_page("puntos_colineales")`: al hacer clic se muestra esa página.
- **Línea 72:** se guarda el botón en el diccionario.
- **Líneas 74-87:** lo mismo para el botón «Módulo Futuro».

6.5 Resaltar el botón activo

main.py · líneas 90-106

```
90 def on_page_changed(page_name):
91     for name, btn in nav_buttons.items():
92         if name == page_name:
93             btn.configure(
94                 fg_color=["#3B82F6", "#2563EB"],
95                 text_color="FFFFFF",
96                 font=ctk.CTkFont(size=13, weight="bold")
97             )
98         else:
99             btn.configure(
100                 fg_color="transparent",
101                 text_color=("gray20", "gray80"),
102                 font=ctk.CTkFont(size=13, weight="normal")
103             )
104
105
106 register_nav_listener(on_page_changed)
```

`on_page_changed` recibe el nombre de la página que se acaba de mostrar. Recorre el diccionario `nav_buttons` con `.items()`, que entrega cada par (nombre, botón):

- Si el nombre coincide con la página activa, pinta el botón de azul (`["#3B82F6", "#2563EB"]` = azul en claro / azul en oscuro), con texto blanco y en negritas.
- Si no, lo deja transparente, con texto gris y peso normal.

Línea 106: `register_nav_listener(on_page_changed)` «suscribe» esta función en `index.py`. Desde ese momento, cada vez que `show_page()` cambie de página, llamará automáticamente a `on_page_changed`. Es el patrón observador (sección 2.8).

6.6 Interruptor de tema

main.py · líneas 108-130

```
108 # Pie de barra lateral: Switch con icono nativo
109 footer_frame = ctk.CTkFrame(sidebar, fg_color="transparent")
110 footer_frame.pack(side="bottom", fill="x", padx=16, pady=20)
111
112
113 def toggle_theme():
114     if theme_switch.get() == 1:
115         ctk.set_appearance_mode("Dark")
116         theme_switch.configure(text=" Modo Oscuro")
117     else:
118         ctk.set_appearance_mode("Light")
119         theme_switch.configure(text=" Modo Claro")
120     notify_theme_to_plot()
121
122
123 theme_switch = ctk.CTkSwitch(
124     footer_frame,
```

```

125     text="  Modo Oscuro",
126     font=ctk.CTkFont(size=12, weight="bold"),
127     command=toggle_theme
128 )
129 theme_switch.pack(anchor="w")
130 theme_switch.select()

```

- **Líneas 109-110:** un marco transparente pegado al fondo de la barra lateral (`side="bottom"`).
- **Líneas 113-120 — `toggle_theme()` :** se ejecuta cada vez que el usuario mueve el interruptor. Si quedó encendido (`get() == 1`) pone modo oscuro; si no, modo claro, y actualiza el texto del interruptor. CustomTkinter cambia solo los colores de todos sus widgets, pero **Matplotlib no se entera**; por eso la línea 120 llama a `notify_theme_to_plot()` , que avisa a la gráfica para que se vuelva a dibujar con los colores correctos.
- **Líneas 123-128:** crea el interruptor y le asigna `toggle_theme` como acción.
- **Línea 130:** `select()` lo deja encendido al inicio, coherente con el modo oscuro de la línea 12.

6.7 Contenedor de páginas y arranque

main.py · líneas 132-143

```

132 # 4. Contenedor Principal de Páginas
133 container = ctk.CTkFrame(root, corner_radius=0, fg_color="transparent")
134 container.grid(row=0, column=1, sticky="nsew")
135 container.rowconfigure(0, weight=1)
136 container.columnconfigure(0, weight=1)
137
138 # Inicializar páginas
139 init_pages(container)
140 on_page_changed("puntos_colineales")
141
142 if __name__ == "__main__":
143     root.mainloop()

```

- **Líneas 133-136:** un marco transparente en la columna 1 de la ventana. Dentro de él tendrá una cuadrícula de una sola celda (fila 0, columna 0) que se estira. Ahí vivirán las páginas.
- **Línea 139:** `init_pages(container)` construye las dos páginas dentro del contenedor (sección 7).
- **Línea 140:** marca «Puntos Colineales» como botón activo.
- **Líneas 142-143:** si el archivo se ejecutó directamente, inicia el ciclo de eventos. A partir de aquí el programa espera acciones del usuario hasta que se cierre la ventana.

7. pages/index.py — el cambio de páginas

Este archivo decide qué página se ve. Su truco: **todas las páginas se colocan en la misma celda** de la cuadrícula, una encima de otra como hojas apiladas, y para «cambiar de página» solo se trae al frente la que se quiere ver.

pages/index.py · líneas 1-14

```
1 """
2 Gestor de páginas y navegación de la aplicación en CustomTkinter.
3 """
4 from pages.Futuro import future_page
5 from pages.puntos_colineales import create_puntos_colineales_page
6
7 _pages = {}
8 _current_page = None
9 _nav_listeners = []
10
11
12 def register_nav_listener(callback):
13     if callback not in _nav_listeners:
14         _nav_listeners.append(callback)
```

- **Líneas 4-5:** importa las dos funciones que construyen las páginas.
- **Línea 7:** `_pages`: diccionario nombre → marco de la página.
- **Línea 8:** `_current_page`: nombre de la página visible; al inicio no hay ninguna (`None`).
- **Línea 9:** `_nav_listeners`: lista de funciones que quieren enterarse de los cambios de página.
- **Líneas 12-14 — `register_nav_listener`:** agrega una función a la lista, pero solo si no estaba ya (`not in`), para no llamarla dos veces.

pages/index.py · líneas 17-27

```
17 def init_pages(container):
18     global _pages, _current_page
19     _pages.update({
20         "puntos_colineales": create_puntos_colineales_page(container),
21         "futuro": future_page(container),
22     })
23
24     for page in _pages.values():
25         page.grid(row=0, column=0, sticky="nsew")
26
27     show_page("puntos_colineales")
```

- **Línea 18:** `global` permite modificar las variables globales desde la función.
- **Líneas 19-22:** llama a las funciones constructoras (cada una devuelve un marco ya lleno) y guarda los resultados en el diccionario con `.update()`.
- **Líneas 24-25:** coloca **cada** página en la fila 0, columna 0 del contenedor. Quedan encimadas; se ve la última colocada.

- **Línea 27:** muestra «Puntos Colineales» para que sea la visible al arrancar.

pages/index.py · líneas 30-44

```
30 def show_page(page_name):
31     global _current_page
32     page = _pages.get(page_name)
33     if page:
34         page.tkraise()
35         _current_page = page_name
36         for cb in _nav_listeners:
37             try:
38                 cb(page_name)
39             except Exception:
40                 pass
41
42
43 def get_current_page():
44     return _current_page
```

- **Línea 32:** busca el marco de la página con `.get()`; si el nombre no existe, obtiene `None` y el `if` de la línea 33 no hace nada.
- **Línea 34:** `tkraise()` trae ese marco al frente de la pila: es lo que hace que la página «aparezca».
- **Línea 35:** recuerda cuál es la página actual.
- **Líneas 36-40:** llama a cada función suscrita pasándole el nombre de la página (en la práctica, `on_page_changed` de `main.py`). El `try/except` evita que un error en un suscriptor rompa la navegación.
- **Líneas 43-44 — `get_current_page`:** devuelve el nombre de la página actual. La usan las pruebas.

OBSERVACIÓN

Cambiar de página **no borra nada**: la página de Puntos Colineales sigue existiendo detrás, con sus datos, su tabla y su gráfica. Al regresar, todo está como se dejó.

8. pages/icons.py — los iconos

En lugar de usar archivos de imagen, la app **dibuja** sus iconos con código usando Pillow. Los diseños imitan el estilo de la colección de iconos «Lucide», que se diseñan sobre una cuadrícula de 24 × 24 unidades.

8.1 Dibujar una figura

pages/icons.py · líneas 1-27

```
1 import math
2 import customtkinter as ctk
3 from PIL import Image, ImageDraw
4
5 _pil_cache = {}
6
7
8 def _draw_lucide_shape(name, draw, s, rgba):
9     scale = s / 24.0
10    lw = max(1, round(2.0 * scale))
11
12    def pt(x, y):
13        return (x * scale, y * scale)
14
15    def line(pts):
16        scaled_pts = [pt(x, y) for (x, y) in pts]
17        for i in range(len(scaled_pts) - 1):
18            draw.line([scaled_pts[i], scaled_pts[i + 1]], fill=rgba, width=lw, joint="curve")
19
20    def circle(cx, cy, r, fill=None, outline=rgba, width=lw):
21        box = [pt(cx - r, cy - r), pt(cx + r, cy + r)]
22        draw.ellipse(box, fill=fill, outline=outline, width=width)
23
24    if name == "trending-up" or name == "chart":
25        line([(23, 6), (13.5, 15.5), (8.5, 10.5), (1, 18)])
26        line([(17, 6), (23, 6), (23, 12)])
27
```

- **Línea 5:** `_pil_cache`, un diccionario que funciona como **caché**: guarda iconos ya dibujados para no dibujarlos otra vez.
- **Línea 8 —** `_draw_lucide_shape(name, draw, s, rgba)`: dibuja el icono llamado `name` con el «lápiz» `draw` sobre una imagen de `s × s` píxeles, con el color `rgba`.
- **Línea 9:** `scale` convierte unidades de la cuadrícula de 24 a píxeles reales. Si la imagen mide 96 px, `scale = 4`: la coordenada 12 de la cuadrícula está en el píxel 48.
- **Línea 10:** grosor de línea: 2 unidades escaladas, redondeado, y como mínimo 1 píxel.
- **Líneas 12-22:** tres funciones auxiliares internas:
 - `pt(x, y)` convierte un punto de la cuadrícula a píxeles;
 - `line(pts)` recibe una lista de puntos y traza segmentos que unen cada punto con el siguiente (el punto `i` con el `i + 1`); `joint="curve"` redondea las uniones;

- `circle(cx, cy, r, ...)` dibuja un círculo de centro (cx, cy) y radio r, calculando la «caja» que lo contiene: de (cx - r, cy - r) a (cx + r, cy + r).
- **Líneas 24-26:** primer icono, «trending-up» (flecha que sube): una línea en zigzag y una punta de flecha. Las coordenadas (y) crecen hacia abajo en imágenes: y = 6 está arriba y y = 18 abajo.

El resto de la función (líneas 28-95) es una cadena de `elif`, uno por icono, que combina líneas, círculos, arcos (`draw.arc`) y polígonos rellenos (`draw.polygon`):



Figura 3. Los iconos que usa la app, dibujados por `icons.py`: logo, trending-up, sparkles, rocket, play, rotate-ccw, table y grid-math. El archivo también define «sun» y «moon», que actualmente no se usan. Si se pide un nombre desconocido (línea 94), dibuja un círculo relleno.

EJEMPLO · EL ICONO «MOON» (LÍNEAS 81-92)

Construye la luna como un polígono: recorre un arco exterior de -80° a 80° (cada 10°) y un arco interior de regreso, calculando cada punto con coseno y seno: $x = cx + r \cdot \cos(\text{ángulo})$, $y = cy + r \cdot \sin(\text{ángulo})$. Esa es la forma paramétrica de un círculo.

8.2 Suavizado por sobremuestreo

pages/icons.py · líneas 98-103

```
98 def _render_pil(name, size, rgba):
99     render_size = size * 4
100     img = Image.new("RGBA", (render_size, render_size), (0, 0, 0, 0))
101     draw = ImageDraw.Draw(img)
102     _draw_lucide_shape(name, draw, render_size, rgba)
103     return img.resize((size, size), Image.Resampling.LANCZOS)
```

Dibujar directamente en un icono pequeño (por ejemplo, 18 px) produce bordes dentados («escalera»), porque cada píxel está completamente pintado o completamente vacío. La solución es el **sobremuestreo** (*supersampling*):

1. **Línea 99:** dibujar a 4 veces el tamaño ($18 \times 4 = 72$ px).
2. **Línea 100:** la imagen empieza completamente transparente (alfa = 0).
3. **Línea 103:** reducirla al tamaño final con el filtro LANCZOS, que promedia los píxeles: los bordes quedan con tonos intermedios y se ven suaves.



Figura 4. El icono «logo» ampliado: a la izquierda, dibujado directo a 24 px; a la derecha, dibujado a 96 px y reducido a 24 px. El segundo tiene bordes suaves.

8.3 La función pública `get_icon`

pages/icons.py · líneas 106-122

```
106 def get_icon(name, size=18, light_color="#1E293B", dark_color="#F8FAFC"):
107     """
108     Retorna un CTKImage fresco creado sobre el intérprete Tkinter activo,
109     reutilizando las imágenes base de Pillow cacheadas en memoria.
110     """
111     key = (name, size, light_color, dark_color)
112     if key not in _pil_cache:
113         def to_rgba(hex_str):
114             hex_str = hex_str.lstrip("#")
115             return (int(hex_str[0:2], 16), int(hex_str[2:4], 16), int(hex_str[4:6], 16), 255)
116
117             img_light = _render_pil(name, size, to_rgba(light_color))
118             img_dark = _render_pil(name, size, to_rgba(dark_color))
119             _pil_cache[key] = (img_light, img_dark)
120
121     img_light, img_dark = _pil_cache[key]
122     return ctk.CTKImage(light_image=img_light, dark_image=img_dark, size=(size, size))
```

- **Línea 111:** una **clave** que identifica el icono de forma única: la tupla (nombre, tamaño, color claro, color oscuro).
- **Línea 112:** si ese icono no está en la caché, hay que dibujarlo.
- **Líneas 113-115 — `to_rgba`:** convierte `"#1E293B"` en `(30, 41, 59, 255)`. Quita el `#` y convierte cada par de caracteres de base 16 a número: `int("1E", 16) = 1×16 + 14 = 30`. El 255 final significa opaco.
- **Líneas 117-119:** dibuja dos versiones (una por tema) y las guarda en la caché.
- **Línea 122:** devuelve un `CTKImage` con ambas versiones. `CustomTkinter` mostrará una u otra según el tema activo, sin que el código tenga que hacer nada más.

9. pages/Futuro.py — la página informativa

pages/Futuro.py · líneas 8-50

```
8 def future_page(container):
9     page = ctk.CTkFrame(container, corner_radius=0, fg_color="transparent")
10
11     # Contenedor central tipo tarjeta
12     card = ctk.CTkFrame(page, corner_radius=16, border_width=1, border_color=("gray80",
"gray30"))
13     card.place(relx=0.5, rely=0.5, anchor="center")
14
15     rocket_icon = get_icon("rocket", size=44, light_color="#2563EB", dark_color="#3B82F6")
16     icon_label = ctk.CTkLabel(card, text="", image=rocket_icon)
17     icon_label.pack(pady=(35, 10))
18
19     title_label = ctk.CTkLabel(
20         card,
21         text="Módulo en Desarrollo",
22         font=ctk.CTkFont(size=20, weight="bold")
23     )
24     title_label.pack(padx=50, pady=(0, 10))
25
26     desc_label = ctk.CTkLabel(
27         card,
28         text="Este espacio está reservado para nuevos algoritmos de computación gráfica:\n\n"
29             "• Rasterización DDA y Bresenham\n"
30             "• Trazado de circunferencias y elipses\n"
31             "• Transformaciones geométricas 2D y curvas paramétricas",
32         font=ctk.CTkFont(size=13),
33         justify="left",
34         text_color=("gray40", "gray70")
35     )
36     desc_label.pack(padx=50, pady=(0, 25))
37
38     badge = ctk.CTkButton(
39         card,
40         text="Próximamente",
41         width=130,
42         height=28,
43         corner_radius=14,
44         state="disabled",
45         fg_color=("gray85", "gray25"),
46         text_color_disabled=("gray40", "gray75")
47     )
48     badge.pack(pady=(0, 35))
49
50     return page
```

- **Línea 9:** la página es un marco transparente dentro del contenedor.
- **Líneas 12-13:** una «tarjeta» con esquinas de 16 px y borde, centrada con `place(relx=0.5, rely=0.5, anchor="center")`: su centro se ubica en el 50 % del ancho y el 50 % del alto.
- **Líneas 15-17:** icono de cohete de 44 px (una etiqueta sin texto, solo imagen).

- **Líneas 19-24:** título «Módulo en Desarrollo».
- **Líneas 26-35:** descripción con los algoritmos planeados. `\n` dentro de un texto significa «salto de línea»; `justify="left"` alinea las líneas a la izquierda.
- **Líneas 38-48:** un botón «Próximamente» **deshabilitado** (`state="disabled"`): no se puede presionar; funciona como una insignia visual.
- **Línea 50:** devuelve el marco para que `index.py` lo coloque.

Esta página no tiene lógica: solo muestra información. Para salir de ella, el usuario usa los botones de la barra lateral, que siguen visibles.

10. pages/puntos_colineales.py — el corazón de la app

Es el archivo más grande (676 líneas). Contiene tres cosas: (1) funciones matemáticas puras, incluida la que identifica el caso de trazo; (2) funciones que dibujan la gráfica, y (3) la función que construye la página y reacciona a los botones.

10.1 Importaciones, estado compartido y avisos de cambio de tema

pages/puntos_colineales.py · líneas 1-26

```
1 import math
2 from tkinter import messagebox
3 import customtkinter as ctk
4 from CTkTable import CTkTable
5
6 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
7 from matplotlib.figure import Figure
8
9 from pages.icons import get_icon
10
11 # Almacenamiento de datos actuales para repintado reactivo
12 current_data = {}
13 _plot_callbacks = []
14
15
16 def register_plot_updater(callback):
17     if callback not in _plot_callbacks:
18         _plot_callbacks.append(callback)
19
20
21 def notify_theme_to_plot():
22     for cb in _plot_callbacks:
23         try:
24             cb()
25         except Exception:
26             pass
```

- **Línea 2:** `messagebox` muestra ventanas emergentes de error.
- **Línea 4:** el componente de tabla.
- **Líneas 6-7:** el adaptador de Matplotlib para Tkinter y la clase `Figure`.
- **Línea 12 — `current_data`:** un diccionario global con los datos del último cálculo (puntos, pendiente, ecuación, caso...). Existe para poder **redibujar** la gráfica en cualquier momento (por ejemplo, al cambiar de tema) sin volver a leer las cajas de texto ni recalcular.
- **Línea 13 — `_plot_callbacks`:** lista de funciones que redibujan la gráfica.
- **Líneas 16-18:** `register_plot_updater` suscribe una función a esa lista.

- **Líneas 21-26:** `notify_theme_to_plot` llama a todas las funciones suscritas. `main.py` la invoca al cambiar el tema.

10.2 Funciones matemáticas

Son funciones «puras»: reciben números y devuelven un resultado, sin tocar la interfaz. Eso permite probarlas por separado (sección 11).

pages/puntos_colineales.py · líneas 29-54

```

29 def get_pendiente(x1, y1, x2, y2):
30     pendiente = (y2 - y1) / (x2 - x1)
31     # Evita el "-0" que resulta de dividir 0 entre un número negativo
32     return 0.0 if pendiente == 0 else pendiente
33
34
35 def get_ordenada(x1, y1, pendiente):
36     return y1 - pendiente * x1
37
38
39 def get_ecuacion_recta(x1, y1, pendiente):
40     ordenada = get_ordenada(x1, y1, pendiente)
41     if pendiente == 0:
42         return f"y = {ordenada:g}"
43
44     if pendiente == 1:
45         termino_x = "x"
46     elif pendiente == -1:
47         termino_x = "-x"
48     else:
49         termino_x = f"{pendiente:g}x"
50
51     if ordenada == 0:
52         return f"y = {termino_x}"
53     signo = "+" if ordenada > 0 else "-"
54     return f"y = {termino_x} {signo} {abs(ordenada):g}"

```

- **get_pendiente (29-32):** calcula $m = (Y_2 - Y_1) / (X_2 - X_1)$. La línea 32 corrige un detalle de las computadoras: dividir 0 entre un número negativo da **-0.0** («cero negativo»), que se mostraría como «-0». Si el resultado es cero, se devuelve 0.0 normal.
- **get_ordenada (35-36):** $b = Y_1 - m \cdot X_1$.
- **get_ecuacion_recta (39-54):** arma el texto de la ecuación de la forma más limpia posible:
 - Líneas 41-42: si $m = 0$ (recta horizontal) escribe solo $y = b$, por ejemplo « $y = 2$ », en vez de « $y = 0x + 2$ ».
 - Líneas 44-49: el término de x . Si $m = 1$ escribe « x » (no « $1x$ »); si $m = -1$, « $-x$ »; en otro caso, el número seguido de x , como « $1.5x$ ».
 - Líneas 51-52: si $b = 0$, no agrega nada más: « $y = x$ ».
 - Líneas 53-54: si no, agrega el signo aparte y el valor absoluto de b , para evitar textos como « $y = 2x + -3$ ».

A	B	m	b	Texto que devuelve
(1, 2)	(5, 8)	1.5	0.5	$y = 1.5x + 0.5$
(1, 2)	(6, 2)	0	2	$y = 2$
(0, 0)	(5, 5)	1	0	$y = x$
(0, 3)	(3, 0)	-1	3	$y = -x + 3$
(0, -3)	(1, -1)	2	-3	$y = 2x - 3$

pages/puntos_colineales.py · líneas 93-114

```

93 def get_direccion_x(dx):
94     if dx > 0:
95         return "Izquierda a Derecha"
96     elif dx < 0:
97         return "Derecha a Izquierda"
98     return "Sin cambio horizontal"
99
100
101 def get_direccion_y(dy):
102     if dy > 0:
103         return "Abajo a Arriba"
104     elif dy < 0:
105         return "Arriba a Abajo"
106     return "Sin cambio vertical"
107
108
109 def get_comportamiento_pendiente(m):
110     if m > 0:
111         return "Ascendente"
112     elif m < 0:
113         return "Descendente"
114     return "Horizontal"

```

Función	Qué devuelve	Con A(1, 2), B(5, 8)
<code>get_direccion_x</code> (93-98)	Sentido horizontal según el signo de ΔX ; si $\Delta X = 0$, «Sin cambio horizontal»	$\Delta X = 4 \rightarrow$ « Izquierda a Derecha »
<code>get_direccion_y</code> (101-106)	Sentido vertical según el signo de ΔY ; si $\Delta Y = 0$, «Sin cambio vertical»	$\Delta Y = 6 \rightarrow$ « Abajo a Arriba »
<code>get_comportamiento_pendiente</code> (109-114)	Ascendente / Descendente / Horizontal según el signo de m	$m = 1.5 \rightarrow$ « Ascendente »

IMPORTANTE

`get_pendiente` divide entre $(X_2 - X_1)$ sin revisar si es cero. No hay problema porque `calculate()` solo la llama cuando $\Delta X \neq 0$; los casos con $\Delta X = 0$ (vertical y punto único) se resuelven aparte (sección 10.7).

10.3 Clasificación de los 11 casos de trazo

La función `clasificar_recta(dx, dy)` recibe ΔX y ΔY y devuelve una **tupla de dos textos**: el nombre del caso y un detalle que explica por qué. Los casos se definen en la sección 4.7; aquí se ve cómo el código los distingue.

pages/puntos_colineales.py · líneas 57-90

```
57 def clasificar_recta(dx, dy):
58     """
59     Identifica el caso de trazo según los incrementos  $\Delta X$  y  $\Delta Y$ .
60     Retorna una tupla (caso, detalle).
61     """
62     if dx == 0 and dy == 0:
63         return "Un solo punto", "A y B son iguales: no hay recorrido"
64
65     if dy == 0:
66         if dx > 0:
67             return "Horizontal hacia la derecha", " $\Delta Y = 0$  y  $\Delta X > 0$ : avanza 1 en X por paso"
68         return "Horizontal hacia la izquierda", " $\Delta Y = 0$  y  $\Delta X < 0$ : retrocede 1 en X por paso"
69
70     if dx == 0:
71         if dy > 0:
72             return "Vertical hacia arriba", " $\Delta X = 0$  y  $\Delta Y > 0$ : pendiente indefinida"
73         return "Vertical hacia abajo", " $\Delta X = 0$  y  $\Delta Y < 0$ : pendiente indefinida"
74
75     if dx < 0:
76         # El mismo trazo recorrido de B hacia A va de izquierda a derecha
77         caso_equivalente, _ = clasificar_recta(-dx, -dy)
78         return (
79             "Dibujada al revés (de derecha a izquierda)",
80             f" $\Delta X < 0$ : equivale a «{caso_equivalente}» recorrida de B hacia A"
81         )
82
83     if abs(abs(dx) - abs(dy)) < 1e-9:
84         sentido = "subiendo" if dy > 0 else "bajando"
85         return "Diagonal exacta (45°)", f" $|\Delta X| = |\Delta Y|$ , {sentido}: avanza 1 en X y 1 en Y por paso"
86
87     sentido = "subiendo" if dy > 0 else "bajando"
88     if abs(dy) < abs(dx):
89         return f"Acostada, {sentido} hacia la derecha", " $|\Delta Y| < |\Delta X|$ : avanza 1 en X por paso"
90     return f"Parada, {sentido} hacia la derecha", " $|\Delta Y| > |\Delta X|$ : avanza 1 en Y por paso"
```

El orden de las preguntas importa: cada `return` termina la función, así que una pregunta solo se hace si todas las anteriores fueron «no».

- Líneas 62-63:** si no hay cambio en ninguno de los dos ejes, A y B son el mismo punto.
- Líneas 65-68:** si $\Delta Y = 0$ (y ya sabemos que $\Delta X \neq 0$), la recta es horizontal. El signo de ΔX dice si va a la derecha o a la izquierda.
- Líneas 70-73:** si $\Delta X = 0$ (y $\Delta Y \neq 0$), la recta es vertical. El signo de ΔY dice si sube o baja.

- 4 **Líneas 75-81 — línea al revés:** si ΔX es negativo, el trazo va de derecha a izquierda. Para decir a qué caso «normal» equivale, la función **se llama a sí misma** (recursión) con los signos invertidos, `clasificar_recta(-dx, -dy)`: es el mismo segmento recorrido de B hacia A. El guion bajo en `caso_equivalente`, `_ = ...` es un nombre para «un valor que no me interesa» (el detalle).
- 5 **Líneas 83-85 — diagonal de 45°:** si $|\Delta X| = |\Delta Y|$. No se compara con `==` sino verificando que la diferencia sea menor que `0.000000001` ($1e-9$), porque con decimales dos números «iguales» pueden diferir en la última cifra.
- 6 **Líneas 87-90:** lo que queda son rectas que van hacia la derecha ($\Delta X > 0$). Si $|\Delta Y| < |\Delta X|$ la recta está **acostada** (inclinación menor de 45°); si no, **parada** (mayor de 45°). El signo de ΔY decide «subiendo» o «bajando».

EJEMPLO · A(5, 8) → B(1, 2)

$\Delta X = -4$, $\Delta Y = -6$. No es punto, ni horizontal, ni vertical. $\Delta X < 0$, así que es «**Dibujada al revés (de derecha a izquierda)**». Para el detalle se clasifica (4, 6): $|6| > |4|$ y $6 > 0 \rightarrow$ «Parada, subiendo hacia la derecha». El detalle queda: « $\Delta X < 0$: equivale a «Parada, subiendo hacia la derecha» recorrida de B hacia A».

10.4 Algoritmo de trayectoria (DDA)

pages/puntos_colineales.py · líneas 117-145

```
117 def generar_puntos_trayectoria(x1, y1, x2, y2):
118     dx = x2 - x1
119     dy = y2 - y1
120     pasos = int(max(abs(dx), abs(dy)))
121     puntos = []
122
123     if dx == 0 and dy == 0:
124         return [(0, x1, y1)]
125     if pasos == 0:
126         # Puntos distintos a menos de una unidad: un solo paso de A a B
127         pasos = 1
128
129     step_x = dx / pasos
130     step_y = dy / pasos
131
132     for i in range(pasos + 1):
133         px = x1 + i * step_x
134         py = y1 + i * step_y
135         if abs(px - round(px)) < 1e-7:
136             px = int(round(px))
137         else:
138             px = round(px, 4)
139         if abs(py - round(py)) < 1e-7:
140             py = int(round(py))
141         else:
```

```

142         py = round(py, 4)
143         puntos.append((i, px, py))
144
145     return puntos

```

- 1 **Líneas 118-120:** calcula ΔX , ΔY y el número de pasos = el mayor de $|\Delta X|$ y $|\Delta Y|$. `int()` quita los decimales (por ejemplo, $4.7 \rightarrow 4$) porque el número de pasos debe ser entero.
- 2 **Líneas 123-124:** si A y B son el mismo punto, devuelve una lista con ese único punto y termina.
- 3 **Líneas 125-127:** si los puntos son distintos pero están a menos de una unidad (por ejemplo, de (0, 0) a (0.5, 0.2)), `int()` daría 0 pasos. Se fuerza 1 paso para que la trayectoria tenga A y B, y para no dividir entre cero en la línea siguiente.
- 4 **Líneas 129-130:** cuánto avanzar en X y en Y en cada paso. Pueden ser negativos: así se recorren las líneas hacia la izquierda o hacia abajo.
- 5 **Líneas 132-134:** para cada i desde 0 hasta `pasos` (por eso `range(pasos + 1)`: incluye el último), calcula el punto i .
- 6 **Líneas 135-142 — limpieza de decimales:** las computadoras guardan los decimales de forma aproximada, así que $1 + 3 \times 0.6667$ puede dar 3.0000000000000004 en lugar de 3. Si un valor está a menos de 0.0000001 ($1e-7$) de un entero, se convierte en ese entero; si no, se redondea a 4 decimales.
- 7 **Línea 143:** agrega la tupla (número de paso, x, y) a la lista. **Línea 145:** la devuelve.

EJEMPLO · A(1, 2) → B(5, 8)

$\Delta X = 4$, $\Delta Y = 6 \rightarrow \text{pasos} = \max(4, 6) = 6 \cdot \text{step}_x = 4/6 = 0.6667 \cdot \text{step}_y = 6/6 = 1$

Paso i	Cálculo de X	X	Cálculo de Y	Y
0	$1 + 0 \times 0.6667$	1	$2 + 0 \times 1$	2
1	$1 + 1 \times 0.6667$	1.6667	$2 + 1 \times 1$	3
2	$1 + 2 \times 0.6667$	2.3333	$2 + 2 \times 1$	4
3	$1 + 3 \times 0.6667$	3	$2 + 3 \times 1$	5
4	$1 + 4 \times 0.6667$	3.6667	$2 + 4 \times 1$	6
5	$1 + 5 \times 0.6667$	4.3333	$2 + 5 \times 1$	7
6	$1 + 6 \times 0.6667$	5	$2 + 6 \times 1$	8

Resultado: 7 puntos (6 pasos + el punto inicial). Y avanza exactamente 1 cada paso porque es el eje que más cambia (es una recta «parada»).

Caso	A → B	Pasos	step_x, step_y	Puntos generados
Vertical hacia abajo	(3, 7) → (3, 1)	6	0, -1	(3, 7), (3, 6), ..., (3, 1)
Horizontal a la izquierda	(6, 2) → (1, 2)	5	-1, 0	(6, 2), (5, 2), ..., (1, 2)
Acostada, bajando	(0, 0) → (3, -2)	3	1, -0.6667	(0, 0), (1, -0.6667), (2, -1.3333), (3, -2)
Un solo punto	(2, 3) → (2, 3)	—	—	(2, 3)

10.5 Dibujo del plano

plot_points — los extremos de la recta

pages/puntos_colineales.py · líneas 148-152

```

148 def plot_points(x1, x2, y1, pendiente, limit):
149     ordenada = get_ordenada(x1, y1, pendiente)
150     x_line = [-limit, limit]
151     y_line = [pendiente * (-limit) + ordenada, pendiente * limit + ordenada]
152     return {"X": x_line, "Y": y_line}

```

Para dibujar una recta «infinita» basta con dos puntos en los bordes visibles del plano. Toma $x = -\text{limit}$ y $x = +\text{limit}$ y calcula su y con la ecuación $y = m \cdot x + b$. Con $m = 1.5$, $b = 0.5$ y $\text{limit} = 10$: los extremos son $(-10, -14.5)$ y $(10, 15.5)$. Devuelve un diccionario con las listas de X y de Y . (El parámetro `x2` se recibe pero no se usa.) Solo sirve para rectas no verticales; la vertical se resuelve en `calculate()`.

setup_ejes — el plano vacío

pages/puntos_colineales.py · líneas 155-181

```

155 def setup_ejes(ax, figure, limit=10, is_dark=True):
156     fig_bg = "#2B2B2B" if is_dark else "#F2F2F2"
157     ax_bg = "#1A1A1A" if is_dark else "#FFFFFF"
158     text_color = "#E0E0E0" if is_dark else "#1A1A1A"
159     subtext = "#9E9E9E" if is_dark else "#616161"
160     grid_color = "#2E2E2E" if is_dark else "#E0E0E0"
161     origin_color = "#616161" if is_dark else "#9E9E9E"
162     spine_color = "#444444" if is_dark else "#D0D0D0"
163
164     figure.patch.set_facecolor(fig_bg)
165     ax.set_facecolor(ax_bg)
166
167     ax.set_title("Plano Cartesiano", color=text_color, fontsize=12, fontweight="bold", pad=12)
168     ax.set_xlabel("Eje X", color=subtext, fontsize=10, labelpad=6)
169     ax.set_ylabel("Eje Y", color=subtext, fontsize=10, labelpad=6)
170
171     ax.tick_params(colors=subtext, labelsize=9)
172     for spine in ax.spines.values():
173         spine.set_color(spine_color)
174         spine.set_linewidth(0.8)
175
176     ax.grid(True, linestyle="--", linewidth=0.6, color=grid_color, alpha=0.7)
177     ax.axhline(0, color=origin_color, linewidth=1.0, linestyle="-")

```

```

178     ax.axvline(0, color=origin_color, linewidth=1.0, linestyle="-")
179     ax.set_xlim(-limit, limit)
180     ax.set_ylim(-limit, limit)
181     ax.set_aspect("equal", adjustable="box")

```

- **Líneas 156-162:** elige 7 colores según `is_dark`. La expresión `A if condición else B` vale A si la condición es verdadera y B si no.
- **Líneas 164-165:** color de fondo del lienzo y del área de dibujo.
- **Líneas 167-169:** título y nombres de los ejes. `pad / labelpad` son márgenes.
- **Líneas 171-174:** color y tamaño de los números de los ejes (*ticks*) y de los bordes del área (*spines*).
- **Línea 176:** cuadrícula punteada (`"--"`), semitransparente (`alpha=0.7`).
- **Líneas 177-178:** dibuja los ejes X y Y que pasan por el origen: una línea horizontal en $y = 0$ (`axhline`) y una vertical en $x = 0$ (`axvline`).
- **Líneas 179-180:** muestra de `-limit` a `+limit` en ambos ejes.
- **Línea 181:** `set_aspect("equal")` hace que una unidad mida lo mismo en X y en Y (escala 1:1). Sin esto, la recta se vería más o menos inclinada de lo que es.

render_plot — dibujar todo

pages/puntos_colineales.py · líneas 184-203

```

184 def render_plot(ax, figure, canvas):
185     is_dark = (ctk.get_appearance_mode().lower() == "dark")
186     line_color = "#3B82F6" if is_dark else "#2563EB"
187     pt_color = "#38BDF8" if is_dark else "#0284C7"
188     pt_a_color = "#34D399" if is_dark else "#059669"
189     pt_b_color = "#A78BFA" if is_dark else "#7C3AED"
190     fig_bg = "#2B2B2B" if is_dark else "#F2F2F2"
191     border_color = "#444444" if is_dark else "#D0D0D0"
192     text_color = "#E0E0E0" if is_dark else "#1A1A1A"
193
194     ax.clear()
195     limit = current_data.get("limit", 10)
196     setup_ejes(ax, figure, limit=limit, is_dark=is_dark)
197
198     if current_data.get("has_data"):
199         points = current_data["points"]
200         table_points = current_data["table_points"]
201         x1, y1 = current_data["x1"], current_data["y1"]
202         x2, y2 = current_data["x2"], current_data["y2"]
203         equation = current_data["equation"]

```

- **Línea 185:** consulta el tema actual. Como lo consulta cada vez que dibuja, basta con volver a llamar a esta función para adaptarse a un cambio de tema.
- **Líneas 186-192:** colores de la recta, puntos, punto A (verde), punto B (morado), etc.
- **Línea 194:** `ax.clear()` borra todo lo dibujado antes.
- **Líneas 195-196:** redibuja el plano vacío con el límite guardado (10 si no hay datos).

- **Línea 198:** solo si hay un cálculo guardado (`has_data`) dibuja la recta y los puntos; si no, queda el plano vacío.
- **Líneas 199-203:** lee los datos guardados en `current_data`.

pages/puntos_colineales.py · líneas 205-233

```

205         # Recta continua (no existe cuando A y B son el mismo punto)
206         if points:
207             ax.plot(
208                 points["X"], points["Y"],
209                 color=line_color,
210                 linewidth=2.4,
211                 label=f"Recta: {equation}"
212             )
213
214         # Puntos de trayectoria
215         if table_points and len(table_points) > 2:
216             t_x = [pt[1] for pt in table_points]
217             t_y = [pt[2] for pt in table_points]
218             ax.plot(
219                 t_x, t_y,
220                 marker="o",
221                 markersize=4.5,
222                 color=pt_color,
223                 linestyle="None",
224                 alpha=0.6,
225                 label="Puntos de trayectoria"
226             )
227
228         # Punto A y Punto B (si coinciden se dibuja uno solo)
229         mismo_punto = (x1 == x2 and y1 == y2)
230         texto_a = f"A = B ({x1:g}, {y1:g})" if mismo_punto else f"A ({x1:g}, {y1:g})"
231         ax.plot([x1], [y1], marker="o", markersize=9, color=pt_a_color, linestyle="None",
label=texto_a)
232         if not mismo_punto:
233             ax.plot([x2], [y2], marker="s", markersize=9, color=pt_b_color, linestyle="None",
label=f"B ({x2:g}, {y2:g})")

```

- **Líneas 206-212:** si hay extremos de recta (`points` no es `None`), los une con una línea. `ax.plot(listaX, listaY)` dibuja una línea que pasa por esos puntos; `label` es el texto de la leyenda. Cuando A y B son el mismo punto, `points` vale `None` y no se dibuja ninguna recta.
- **Líneas 215-226:** si hay más de 2 puntos de trayectoria, los dibuja como círculos pequeños (`marker="o"`) **sin línea** que los una (`linestyle="None"`). Con 2 puntos o menos no tendría sentido, porque serían A y B, que se dibujan aparte.
- **Línea 229:** `mismo_punto` es `True` si A y B coinciden.
- **Línea 230:** el texto de A: «A = B (2, 3)» si coinciden, o «A (1, 2)» si no.
- **Líneas 231-233:** dibuja A como círculo grande. B (un cuadrado, `marker="s"`) solo se dibuja si es distinto de A, para no encimar dos marcadores.

pages/puntos_colineales.py · líneas 235-274

```
235         # Anotaciones
236         bbox_a = dict(boxstyle="round,pad=0.35", fc=fig_bg, ec=pt_a_color, lw=1.2, alpha=0.95)
237         ax.annotate(
238             texto_a,
239             (x1, y1),
240             textcoords="offset points",
241             xytext=(0, 14),
242             ha="center",
243             fontsize=9,
244             fontweight="bold",
245             color=pt_a_color,
246             bbox=bbox_a
247         )
248
249         if not mismo_punto:
250             bbox_b = dict(boxstyle="round,pad=0.35", fc=fig_bg, ec=pt_b_color, lw=1.2,
alpha=0.95)
251             ax.annotate(
252                 f"B ({x2:g}, {y2:g})",
253                 (x2, y2),
254                 textcoords="offset points",
255                 xytext=(0, 14),
256                 ha="center",
257                 fontsize=9,
258                 fontweight="bold",
259                 color=pt_b_color,
260                 bbox=bbox_b
261             )
262
263         legend = ax.legend(
264             loc="upper left",
265             fontsize=8.5,
266             framealpha=0.88,
267             facecolor=fig_bg,
268             edgecolor=border_color
269         )
270         for text in legend.get_texts():
271             text.set_color(text_color)
272
273     figure.tight_layout()
274     canvas.draw()
```

- **Líneas 236-247:** `ax.annotate` escribe la etiqueta de A 14 puntos por encima del punto, dentro de un recuadro redondeado de su color.
- **Líneas 249-261:** la etiqueta de B, solo si es distinto de A.
- **Líneas 263-271:** crea la leyenda en la esquina superior izquierda y pinta sus textos con el color del tema.
- **Línea 273:** `tight_layout()` ajusta márgenes para que nada quede cortado. **Línea 274:** `canvas.draw()` muestra el resultado en la ventana.

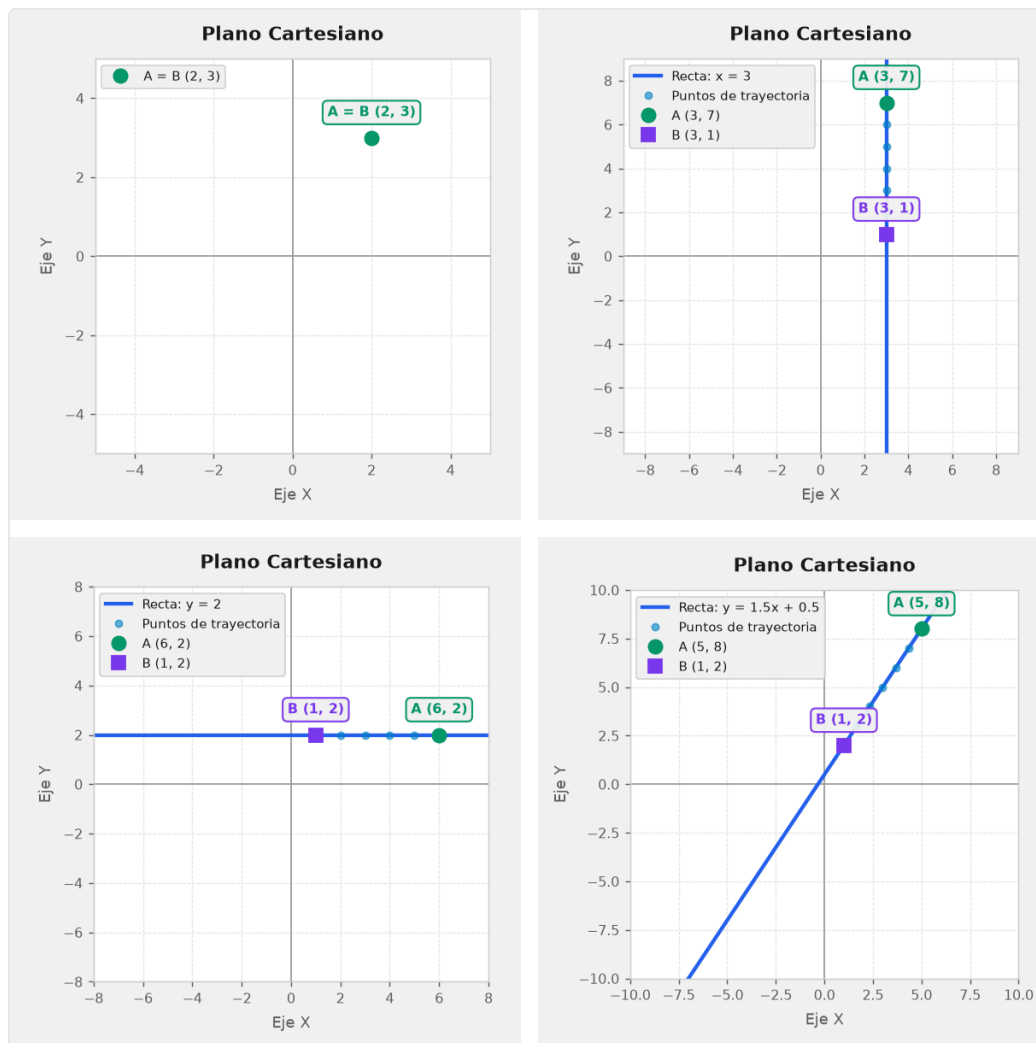


Figura 5. Casos especiales dibujados por `render_plot()` : un solo punto, vertical hacia abajo, horizontal hacia la izquierda y línea dibujada al revés.

10.6 Construcción de la pantalla

`create_puntos_colineales_page(container)` (línea 277) construye toda la página. Aquí se resume por bloques; los números de línea te permiten ubicar cada parte.

Líneas	Bloque	Qué crea
278	Página	Marco transparente <code>page</code> que contendrá todo.
280-299	Encabezado	Título «Trazado y Análisis de Rectas» y subtítulo gris.
301-340	Formulario	Tarjeta con los grupos «Punto A (Origen)» y «Punto B (Destino)». Cada grupo usa <code>grid</code> : fila 0 el título (ocupa 4 columnas), fila 1 las etiquetas X/Y y las cajas <code>entryXA</code> , <code>entryYA</code> , <code>entryXB</code> , <code>entryYB</code> (65 px de ancho, texto centrado).
342-374	Botones	<code>btn_calc</code> «Calcular Trayectoria» (azul, icono play) y <code>btn_clear</code> «Limpiar» (transparente con borde).
376-423	Tarjetas	Cuatro tarjetas en una fila: Pendiente, Ecuación, Incremento X, Incremento Y. Cada una tiene un título, un valor grande (<code>val_..._num</code>) y dos líneas de detalle (<code>val_..._sub1</code> , <code>val_..._sub2</code>).
425-432	Caso detectado	Franja que ocupa las 4 columnas, con el nombre del caso (<code>val_caso</code>) y su explicación (<code>val_caso_sub</code>).
434-487	Tabla	Tarjeta de 250 px con título «Puntos Discretos», contador <code>tbl_count</code> , área desplazable y la tabla.
489-516	Plano	Tarjeta con título «Visualizador Geométrico», la figura de Matplotlib y su lienzo.
518-668	Lógica	Funciones internas <code>calculate</code> , <code>clear</code> y <code>update_plot</code> .
670-676	Inicio	Valores iniciales y <code>return page</code> .

Los botones llaman a funciones definidas después

pages/puntos_colineales.py · líneas 346-357

```

346     icon_play = get_icon("play", size=15, light_color="#FFFFFF", dark_color="#FFFFFF")
347     btn_calc = ctk.CTkButton(
348         actions_box,
349         text="  Calcular Trayectoria",
350         image=icon_play,
351         compound="left",
352         height=34,
353         corner_radius=8,
354         font=ctk.CTkFont(size=12, weight="bold"),
355         command=lambda: calculate()
356     )
357     btn_calc.pack(side="left", padx=(0, 8), pady=6)

```

La línea 355 usa `lambda: calculate()` y no `command=calculate`. ¿Por qué? Porque en la línea 355, `calculate` **todavía no existe**: se define hasta la línea 519. La `lambda` retrasa la búsqueda del nombre hasta el momento del clic, cuando la función ya fue creada.

Las tarjetas de métricas y la franja del caso

pages/puntos_colineales.py · líneas 376-390

```
376     # --- 3. TARJETA DE MÉTRICAS (DATOS SEPARADOS Y DETALLADOS) ---
377     metrics_card = ctk.CTkFrame(page, corner_radius=12, border_width=1, border_color=
("gray85", "gray25"))
378     metrics_card.pack(fill="x", padx=20, pady=(0, 8))
379     metrics_card.columnconfigure((0, 1, 2, 3), weight=1, uniform="stat")
380
381     # Tile 1: Pendiente
382     tile_m = ctk.CTkFrame(metrics_card, corner_radius=10, fg_color=("gray92", "gray20"))
383     tile_m.grid(row=0, column=0, sticky="nsew", padx=6, pady=8)
384     ctk.CTkLabel(tile_m, text="PENDIENTE (m)", font=ctk.CTkFont(size=9, weight="bold"),
text_color="gray", anchor="w").pack(anchor="w", padx=12, pady=(6, 0))
385     val_m_num = ctk.CTkLabel(tile_m, text="--", font=ctk.CTkFont(size=17, weight="bold"),
anchor="w")
386     val_m_num.pack(anchor="w", padx=12, pady=(1, 0))
387     val_m_sub1 = ctk.CTkLabel(tile_m, text="Comportamiento: --", font=ctk.CTkFont(size=10),
text_color=("gray30", "gray70"), anchor="w")
388     val_m_sub1.pack(anchor="w", padx=12)
389     val_m_sub2 = ctk.CTkLabel(tile_m, text="Ángulo: --", font=ctk.CTkFont(size=10),
text_color=("gray30", "gray70"), anchor="w")
390     val_m_sub2.pack(anchor="w", padx=12, pady=(0, 6))
```

Línea 379: la tarjeta tiene 4 columnas con el mismo peso y el mismo grupo `uniform="stat"`, lo que obliga a que midan exactamente lo mismo. Cada tarjeta (`tile_...`) va en su columna con `sticky="nsew"` para ocupar toda la celda. La etiqueta del título (línea 384) se crea y se coloca en la misma línea sin guardarla en una variable, porque nunca cambia; las otras sí se guardan porque `calculate()` y `clear()` modifican su texto.

pages/puntos_colineales.py · líneas 425-432

```
425     # Franja: caso de trazo detectado (ocupa las 4 columnas)
426     tile_caso = ctk.CTkFrame(metrics_card, corner_radius=10, fg_color=("gray92", "gray20"))
427     tile_caso.grid(row=1, column=0, colspan=4, sticky="nsew", padx=6, pady=(0, 8))
428     ctk.CTkLabel(tile_caso, text="CASO DETECTADO", font=ctk.CTkFont(size=9, weight="bold"),
text_color="gray", anchor="w").pack(side="left", padx=(12, 10), pady=6)
429     val_caso = ctk.CTkLabel(tile_caso, text="--", font=ctk.CTkFont(size=13, weight="bold"),
anchor="w")
430     val_caso.pack(side="left", pady=6)
431     val_caso_sub = ctk.CTkLabel(tile_caso, text="", font=ctk.CTkFont(size=10), text_color=
("gray30", "gray70"), anchor="w")
432     val_caso_sub.pack(side="left", padx=(12, 12), pady=6)
```

La franja va en la **fila 1** (debajo de las tarjetas, que están en la fila 0) y con `colspan=4` ocupa el ancho de las cuatro. Dentro, sus tres etiquetas se acomodan de izquierda a derecha con `pack(side="left")`: el título gris «CASO DETECTADO», el nombre del caso en negritas y el detalle.

La tabla

pages/puntos_colineales.py · líneas 464-487

```
464     default_table_data = [["Paso", "Coord X", "Coord Y"]]
465     table_holder = {"table": None}
466
467     def render_table(rows_data):
468         if table_holder["table"] is not None:
469             table_holder["table"].destroy()
470
471         ctk_tbl = CtkTable(
472             table_scroll,
473             row=len(rows_data),
474             column=3,
475             values=rows_data,
476             header_color=("#3B82F6", "#1D4ED8"),
477             colors=[("gray95", "#21252D"), ("gray90", "#1A1D24")],
478             hover=True,
479             hover_color=("#BFDDBF", "#1E3A8A"),
480             corner_radius=8,
481             font=ctk.CtkFont(size=11),
482             justify="center"
483         )
484         ctk_tbl.pack(fill="x", expand=True)
485         table_holder["table"] = ctk_tbl
486
487     render_table(default_table_data)
```

- **Línea 464:** los datos iniciales: solo la fila de encabezado. Es una **lista de listas**: cada lista interior es una fila.
- **Línea 465:** `table_holder` es un diccionario con la tabla actual. Se usa un diccionario (y no una variable simple) porque una función interna no puede *reassignar* una variable de la función externa sin la palabra `nonlocal`, pero sí puede *modificar el contenido* de un diccionario.
- **Líneas 467-485 — `render_table(rows_data)`:** si ya había una tabla, la destruye (468-469); crea una nueva con tantas filas como datos y 3 columnas, con encabezado azul, filas de colores alternados y resaltado al pasar el mouse (`hover`); la coloca y la guarda. Reconstruir la tabla completa es más simple que editar celda por celda.
- **Línea 487:** dibuja la tabla vacía inicial.

El plano

pages/puntos_colineales.py · líneas 510-516

```
510     figure = Figure(figsize=(7, 5), dpi=100)
511     ax = figure.add_subplot(111)
512     setup_ejes(ax, figure, limit=10, is_dark=(ctk.get_appearance_mode().lower() == "dark"))
513
514     canvas = FigureCanvasTkAgg(figure, master=graph_card)
515     canvas.get_tk_widget().pack(fill="both", expand=True, padx=10, pady=(0, 10))
516     canvas.draw()
```


Crea una figura de 7×5 pulgadas a 100 puntos por pulgada, sus ejes, dibuja el plano vacío, la envuelve en un widget de Tkinter y la coloca con `fill="both"`, `expand=True` para que ocupe todo el espacio disponible.

10.7 La función `calculate()` — paso a paso

Se ejecuta al presionar «Calcular Trayectoria». Es la lógica principal de la app. Funciona con **cualquier par de puntos numéricos**, incluidos la recta vertical y el punto único.

Paso 1: leer y validar las entradas

pages/puntos_colineales.py · líneas 519-531

```
519     def calculate():
520         try:
521             x1 = float(entryXA.get().strip())
522             y1 = float(entryYA.get().strip())
523             x2 = float(entryXB.get().strip())
524             y2 = float(entryYB.get().strip())
525             if x1.is_integer(): x1 = int(x1)
526             if y1.is_integer(): y1 = int(y1)
527             if x2.is_integer(): x2 = int(x2)
528             if y2.is_integer(): y2 = int(y2)
529         except ValueError:
530             messagebox.showerror("Dato Inválido", "Por favor ingresa coordenadas numéricas válidas.")
531         return
```

- `entryXA.get()` devuelve el texto escrito, por ejemplo " 5 ". `.strip()` le quita espacios al inicio y al final: "5". `float()` lo convierte a número decimal: 5.0.
- Líneas 525-528: si el número no tiene parte decimal (`is_integer()`), se convierte a entero para que se muestre «5» y no «5.0».
- Si cualquiera de las cuatro conversiones falla (por ejemplo, el usuario escribió «abc» o dejó la caja vacía), `float()` lanza `ValueError`; el `except` muestra una ventana de error y `return` termina la función sin calcular nada. Es la **única** situación en la que no se calcula.

Paso 2: incrementos, sentido, caso y tamaño del plano

pages/puntos_colineales.py · líneas 533-543

```
533     delta_x = x2 - x1
534     delta_y = y2 - y1
535     dir_x = get_direccion_x(delta_x)
536     dir_y = get_direccion_y(delta_y)
537     caso, caso_detalle = clasificar_recta(delta_x, delta_y)
538
539     max_val = max(abs(x1), abs(y1), abs(x2), abs(y2))
540     if max_val == 0:
541         max_val = 10
542     margin = max(2, int(max_val * 0.15) + 1)
543     limit = max_val + margin
```

- **533-536:** ΔX , ΔY y sus sentidos.
- **537:** clasifica el trazo (sección 10.3). La tupla que devuelve se «desempaqueta» en dos variables: `caso` y `caso_detalle`.
- **539-543 — tamaño del plano:** se busca la coordenada más lejana del origen (`max_val`) y se le suma un margen del 15 % (como mínimo 2) para que A y B no queden pegados al borde. Si todas las coordenadas fueran 0, se usa 10. Ejemplo: $\text{max_val} = 8, 8 \times 0.15 = 1.2 \rightarrow \text{int} = 1 \rightarrow 1 + 1 = 2 \rightarrow \text{margen} = \text{máx}(2, 2) = 2 \rightarrow \text{limit} = 10$, así que el plano va de -10 a 10 .

Paso 3: tres caminos según el tipo de trazo

pages/puntos_colineales.py · líneas 545-566

```

545     if delta_x == 0 and delta_y == 0:
546         # Un solo punto: no hay recta, pendiente ni ángulo
547         pendiente = ordenada = angulo = None
548         comportamiento = "Sin dirección (un solo punto)"
549         equation = f"Punto ({x1:g}, {y1:g})"
550         points = None
551     elif delta_x == 0:
552         # Recta vertical: la pendiente es indefinida (división entre cero)
553         pendiente = ordenada = None
554         comportamiento = "Vertical"
555         angulo = 90.0
556         equation = f"x = {x1:g}"
557         points = {"X": [x1, x1], "Y": [-limit, limit]}
558     else:
559         pendiente = get_pendiente(x1, y1, x2, y2)
560         ordenada = get_ordenada(x1, y1, pendiente)
561         comportamiento = get_comportamiento_pendiente(pendiente)
562         angulo = math.degrees(math.atan(pendiente))
563         equation = get_ecuacion_recta(x1, y1, pendiente)
564         points = plot_points(x1, x2, y1, pendiente, limit)
565
566     table_points = generar_puntos_trayectoria(x1, y1, x2, y2)

```

La escritura `pendiente = ordenada = angulo = None` asigna el mismo valor a varias variables a la vez.

Camino	Condición	Resultado
Un solo punto (545-550)	$\Delta X = 0$ y $\Delta Y = 0$	No hay recta: pendiente, ordenada y ángulo valen <code>None</code> ; la «ecuación» es el texto «Punto (2, 3)»; no hay extremos de recta (<code>points = None</code>).
Vertical (551-557)	$\Delta X = 0, \Delta Y \neq 0$	La pendiente sería una división entre cero, así que no se calcula (<code>None</code>) y tampoco existe ordenada (una recta vertical no cruza el eje Y, salvo que sea el propio eje). Ángulo = 90° . Ecuación « $x = 3$ ». Los extremos son ($X_1, -\text{limit}$) y ($X_1, +\text{limit}$): una línea vertical de borde a borde.
Cualquier otro (558-564)	$\Delta X \neq 0$	Pendiente, ordenada, comportamiento, ángulo en grados, ecuación (sección 10.2) y extremos con <code>plot_points</code> .

Línea 566: en los tres casos se generan los puntos de la trayectoria.

Paso 4: guardar el resultado

pages/puntos_colineales.py · líneas 568-588

```
568         current_data.clear()
569         current_data.update({
570             "has_data": True,
571             "x1": x1,
572             "y1": y1,
573             "x2": x2,
574             "y2": y2,
575             "pendiente": pendiente,
576             "ordenada": ordenada,
577             "comportamiento": comportamiento,
578             "angulo": angulo,
579             "equation": equation,
580             "caso": caso,
581             "delta_x": delta_x,
582             "delta_y": delta_y,
583             "dir_x": dir_x,
584             "dir_y": dir_y,
585             "limit": limit,
586             "points": points,
587             "table_points": table_points,
588         })
```

Vacía `current_data` y guarda ahí todos los resultados, incluido el caso. `"has_data": True` indica a `render_plot` que hay algo que dibujar.

Paso 5: mostrar los resultados

pages/puntos_colineales.py · líneas 590-631

```
590         # Actualizar datos separados de manera clara en las 4 tarjetas
591         # 1. Pendiente
592         val_m_num.configure(text=f"{pendiente:g}" if pendiente is not None else "Indefinida")
593         val_m_sub1.configure(text=f"Sentido: {comportamiento}")
594         val_m_sub2.configure(text=f"Ángulo: {angulo:.1f}°" if angulo is not None else "Ángulo:
--")
595
596         # 2. Ecuación
597         val_eq_num.configure(text=equation)
598         if ordenada is not None:
599             val_eq_sub1.configure(text=f"Ordenada (b): {ordenada:g}")
600             val_eq_sub2.configure(text="Forma: y = mx + b")
601         elif points:
602             val_eq_sub1.configure(text="Ordenada (b): no existe")
603             val_eq_sub2.configure(text="Forma: x = constante")
604         else:
605             val_eq_sub1.configure(text="Ordenada (b): no existe")
606             val_eq_sub2.configure(text="Forma: punto aislado")
607
608         # 3. Incremento X
```

```

609     val_dx_num.configure(text=f"{delta_x:+g}" if delta_x != 0 else "0")
610     val_dx_sub1.configure(text=f"Sentido: {dir_x}")
611     val_dx_sub2.configure(text=f"Magnitud: |ΔX| = {abs(delta_x):g}")
612
613     # 4. Incremento Y
614     val_dy_num.configure(text=f"{delta_y:+g}" if delta_y != 0 else "0")
615     val_dy_sub1.configure(text=f"Sentido: {dir_y}")
616     val_dy_sub2.configure(text=f"Magnitud: |ΔY| = {abs(delta_y):g}")
617
618     # 5. Caso detectado
619     val_caso.configure(text=caso)
620     val_caso_sub.configure(text=caso_detalle)
621
622     # Llenar CtkTable
623     new_table_data = [["Paso", "Coord X", "Coord Y"]]
624     for paso, px, py in table_points:
625         new_table_data.append([str(paso), f"{px:g}", f"{py:g}"])
626     render_table(new_table_data)
627
628     tbl_count.configure(text=f"{len(table_points)} puntos calculados")
629
630     # Repintar gráfica
631     render_plot(ax, figure, canvas)

```

- **592-594 — Pendiente:** si no existe (None) muestra «Indefinida»; el ángulo se muestra con un decimal o «--» si no existe. La expresión `X if condición else Y` elige el texto.
- **597-606 — Ecuación:** si hay ordenada, la forma es « $y = mx + b$ »; si no la hay pero sí hay recta (`elif points`), es vertical: « $x = \text{constante}$ »; si no hay recta, es un «punto aislado».
- **609-616 — ΔX y ΔY:** `:+g` fuerza el signo («+4»), salvo cuando vale 0, que se muestra como «0».
- **619-620:** escribe el caso y su detalle en la franja.
- **623-626:** arma las filas de la tabla: encabezado + una fila por punto, convirtiendo cada número a texto. Luego reconstruye la tabla.
- **628:** actualiza el contador («7 puntos calculados»). **631:** redibuja la gráfica.

LO QUE VE EL USUARIO

	A(1, 2) → B(5, 8)	A(3, 7) → B(3, 1)	A(2, 3) → B(2, 3)
Pendiente	1.5 · Ascendente · 56.3°	Indefinida · Vertical · 90.0°	Indefinida · Sin dirección · --
Ecuación	$y = 1.5x + 0.5$ · $b = 0.5$	$x = 3$ · b no existe	Punto (2, 3) · punto aislado
ΔX	+4 · Izquierda a Derecha	0 · Sin cambio horizontal	0 · Sin cambio horizontal
ΔY	+6 · Abajo a Arriba	-6 · Arriba a Abajo	0 · Sin cambio vertical
Caso	Parada, subiendo hacia la derecha	Vertical hacia abajo	Un solo punto
Tabla	7 puntos	7 puntos	1 punto

10.8 La función clear() y el final de la página

pages/puntos_colineales.py · líneas 633-676

```
633     def clear():
634         entryXA.delete(0, "end")
635         entryYA.delete(0, "end")
636         entryXB.delete(0, "end")
637         entryYB.delete(0, "end")
638
639         val_m_num.configure(text="--")
640         val_m_sub1.configure(text="Comportamiento: --")
641         val_m_sub2.configure(text="Ángulo: --")
642
643         val_eq_num.configure(text="--")
644         val_eq_sub1.configure(text="Ordenada (b): --")
645         val_eq_sub2.configure(text="Forma: Explícita")
646
647         val_dx_num.configure(text="--")
648         val_dx_sub1.configure(text="Sentido: --")
649         val_dx_sub2.configure(text="Magnitud: --")
650
651         val_dy_num.configure(text="--")
652         val_dy_sub1.configure(text="Sentido: --")
653         val_dy_sub2.configure(text="Magnitud: --")
654
655         val_caso.configure(text="--")
656         val_caso_sub.configure(text="")
657
658         render_table(default_table_data)
659         tbl_count.configure(text="0 pasos calculados")
660         current_data.clear()
661
662         render_plot(ax, figure, canvas)
663         entryXA.focus_set()
664
665     def update_plot():
666         render_plot(ax, figure, canvas)
667
668     register_plot_updater(update_plot)
669
670     # Valores iniciales para facilitar la visualización inmediata
671     entryXA.insert(0, "1")
672     entryYA.insert(0, "2")
673     entryXB.insert(0, "5")
674     entryYB.insert(0, "8")
675
676     return page
```

- **634-637:** vacía las cuatro cajas de texto: `delete(0, "end")` borra desde el primer carácter hasta el final.
- **639-656:** regresa las tarjetas a «--» y vacía la franja del caso.
- **658-660:** tabla vacía, contador en cero y `current_data` vacío (así `has_data` ya no existe y `render_plot` no dibuja la recta).

- **662-663:** redibuja el plano vacío y pone el cursor en la caja X_1 (`focus_set`) para que el usuario escriba de inmediato.
- **665-668:** `update_plot` solo redibuja; se suscribe a `_plot_callbacks` para que `notify_theme_to_plot()` la llame al cambiar el tema.
- **671-674:** escribe $A(1, 2)$ y $B(5, 8)$ en las cajas como ejemplo, para que el usuario pueda presionar «Calcular» de inmediato.
- **676:** devuelve la página terminada a `index.py`.

11. test_app.py — las pruebas automáticas

Una **prueba automática** es código que ejecuta otro código y comprueba que el resultado sea el esperado. Si alguien cambia algo y rompe una función, la prueba lo detecta. Se usa `unittest`, el módulo de pruebas incluido en Python.

- Las pruebas se agrupan en una **clase** que hereda de `unittest.TestCase`. (Una clase es una «plantilla» que agrupa funciones relacionadas; aquí solo sirve como contenedor.)
- Cada función cuyo nombre empieza con `test_` es una prueba independiente. `self` es el primer parámetro que Python pasa automáticamente a las funciones de una clase.
- Las **aserciones** comprueban resultados: `assertEqual(a, b)` (a debe ser igual a b), `assertAlmostEqual` (casi igual, tolera errores mínimos de decimales), `assertIn(a, b)` (a debe estar contenido en b), `assertTrue`, `assertFalse`, `assertIsNone`, `assertIsNotNone`. Si una aserción falla, la prueba se marca como fallida.
- `with self.subTest(...)` sirve para revisar muchos casos dentro de una misma prueba: si uno falla, el reporte dice cuál, y los demás se siguen revisando.

Hay **8 pruebas**. Se ejecutan con `python -m unittest test_app.py` y actualmente todas pasan.

11.1 Pruebas de la lógica original

test_app.py · líneas 24-48

```
24     def test_math_logic(self):
25         # Pendiente m = (8 - 2) / (5 - 1) = 6 / 4 = 1.5
26         m = get_pendiente(1, 2, 5, 8)
27         self.assertAlmostEqual(m, 1.5)
28
29         # Ordenada b = 2 - 1.5 * 1 = 0.5
30         b = get_ordenada(1, 2, m)
31         self.assertAlmostEqual(b, 0.5)
32
33         # Ecuación
34         eq = get_ecuacion_recta(1, 2, m)
35         self.assertIn("1.5x", eq)
36         self.assertIn("0.5", eq)
37
38         # Direcciones
39         self.assertEqual(get_direccion_x(4), "Izquierda a Derecha")
40         self.assertEqual(get_direccion_x(-3), "Derecha a Izquierda")
41         self.assertEqual(get_direccion_y(6), "Abajo a Arriba")
42         self.assertEqual(get_direccion_y(-5), "Arriba a Abajo")
43
44         # Trayectoria
45         puntos = generar_puntos_trayectoria(1, 2, 5, 8)
46         self.assertEqual(len(puntos), 7) # max(|4|, |6|) = 6 pasos -> 7 puntos (0 a 6)
47         self.assertEqual(puntos[0], (0, 1, 2))
48         self.assertEqual(puntos[-1], (6, 5, 8))
```

test_math_logic comprueba las funciones matemáticas con A(1, 2), B(5, 8): pendiente 1.5, ordenada 0.5, que la ecuación contenga «1.5x» y «0.5», los cuatro sentidos, y que la trayectoria tenga 7 puntos, empezando en (0, 1, 2) y terminando en (6, 5, 8).

test_customtkinter_appearance_modes (líneas 50-57) cambia entre modo oscuro y claro y verifica que CustomTkinter reporte el modo correcto.

test_navigation (líneas 59-74) crea una ventana oculta (`withdraw()`), construye las páginas y verifica que la inicial sea «puntos_colineales», que se pueda ir a «futuro» y regresar. `destroy()` cierra la ventana al final.

test_app.py · líneas 76-117

```
76     def test_interactive_calculation_and_clear(self):
77         app = ctk.CTk()
78         app.withdraw()
79
80         container = ctk.CTkFrame(app)
81         container.pack()
82         page = create_puntos_colineales_page(container)
83         page.pack()
84
85         # Encontrar botones de calcular y limpiar dentro de la página
86         calc_btn = None
87         clear_btn = None
88
89         def search_buttons(widget):
90             nonlocal calc_btn, clear_btn
91             if isinstance(widget, ctk.CTkButton):
92                 text = widget.cget("text")
93                 if "Calcular" in text:
94                     calc_btn = widget
95                 elif "Limpiar" in text:
96                     clear_btn = widget
97             for child in widget.winfo_children():
98                 search_buttons(child)
99
100        search_buttons(page)
101        self.assertIsNotNone(calc_btn)
102        self.assertIsNotNone(clear_btn)
103
104        # Ejecutar cálculo (valores por defecto 1, 2, 5, 8)
105        calc_btn.invoke()
106        self.assertTrue(current_data.get("has_data"))
107        self.assertEqual(current_data.get("pendiente"), 1.5)
108        self.assertEqual(len(current_data.get("table_points")), 7)
109
110        # Probar notificación de tema al plano
111        notify_theme_to_plot()
112
113        # Ejecutar limpieza
114        clear_btn.invoke()
115        self.assertFalse(current_data.get("has_data", False))
116
117        app.destroy()
```


test_interactive_calculation_and_clear simula a un usuario:

- **89-98** — **search_buttons**: una función **recursiva** (se llama a sí misma) que recorre el árbol de widgets de la página buscando los botones cuyo texto contenga «Calcular» y «Limpiar». `isinstance(widget, ctk.CTkButton)` pregunta si el widget es un botón; `wininfo_children()` devuelve sus hijos. `nonlocal` permite modificar las variables `calc_btn` y `clear_btn` de la función exterior.
- **105**: `invoke()` simula el clic en «Calcular» (con los valores iniciales 1, 2, 5, 8) y después se comprueba que `current_data` tenga datos, pendiente 1.5 y 7 puntos.
- **111**: prueba que avisar del cambio de tema no provoque errores.
- **114-115**: simula «Limpiar» y comprueba que ya no haya datos. `.get("has_data", False)` devuelve `False` porque la clave ya no existe.

11.2 Pruebas de los 11 casos

test_app.py · líneas 120-143

```
120     # (A, B, caso esperado) para los 11 casos de trazo
121     CASOS = [
122         ((2, 3), (2, 3), "Un solo punto"),
123         ((1, 2), (6, 2), "Horizontal hacia la derecha"),
124         ((6, 2), (1, 2), "Horizontal hacia la izquierda"),
125         ((3, 1), (3, 7), "Vertical hacia arriba"),
126         ((3, 7), (3, 1), "Vertical hacia abajo"),
127         ((0, 0), (5, 5), "Diagonal exacta (45°)"),
128         ((0, 0), (8, 3), "Acostada, subiendo hacia la derecha"),
129         ((0, 5), (8, 2), "Acostada, bajando hacia la derecha"),
130         ((0, 0), (3, 8), "Parada, subiendo hacia la derecha"),
131         ((0, 8), (3, 0), "Parada, bajando hacia la derecha"),
132         ((5, 8), (1, 2), "Dibujada al revés (de derecha a izquierda)"),
133     ]
134
135     def test_clasificacion_casos(self):
136         for (x1, y1), (x2, y2), esperado in self.CASOS:
137             with self.subTest(caso=esperado):
138                 caso, _ = clasificar_recta(x2 - x1, y2 - y1)
139                 self.assertEqual(caso, esperado)
140
141         # Diagonal bajando y la equivalencia de la línea al revés
142         self.assertEqual(clasificar_recta(4, -4)[0], "Diagonal exacta (45°)")
143         self.assertIn("Parada, subiendo hacia la derecha", clasificar_recta(-4, -6)[1])
```

CASOS (líneas 121-133) es una lista con un ejemplo de cada caso: el punto A, el punto B y el nombre que debe resultar. Al estar dentro de la clase, todas las pruebas la comparten como `self.CASOS`.

test_clasificacion_casos recorre la lista y comprueba que `clasificar_recta` devuelva el nombre esperado para cada uno. Además revisa que una diagonal que baja también sea «Diagonal exacta (45°)» y que el detalle de una línea al revés mencione su caso equivalente.

test_app.py · líneas 145-169

```
145     def test_trayectorias_casos_especiales(self):
146         # Vertical hacia abajo: X constante, Y baja de 1 en 1
147         puntos = generar_puntos_trayectoria(3, 7, 3, 1)
148         self.assertEqual(len(puntos), 7)
149         self.assertTrue(all(px == 3 for _, px, _ in puntos))
150         self.assertEqual(puntos[-1], (6, 3, 1))
151
152         # Horizontal hacia la izquierda: Y constante, X baja de 1 en 1
153         puntos = generar_puntos_trayectoria(6, 2, 1, 2)
154         self.assertEqual([px for _, px, _ in puntos], [6, 5, 4, 3, 2, 1])
155
156         # Al revés: empieza en A y termina en B
157         puntos = generar_puntos_trayectoria(5, 8, 1, 2)
158         self.assertEqual(puntos[0], (0, 5, 8))
159         self.assertEqual(puntos[-1], (6, 1, 2))
160
161         # Puntos distintos a menos de una unidad: A y B
162         self.assertEqual(len(generar_puntos_trayectoria(0, 0, 0.5, 0.2)), 2)
163
164     def test_ecuaciones_casos(self):
165         self.assertEqual(get_ecuacion_recta(1, 2, get_pendiente(1, 2, 6, 2)), "y = 2")
166         self.assertEqual(get_ecuacion_recta(6, 2, get_pendiente(6, 2, 1, 2)), "y = 2")
167         self.assertEqual(get_ecuacion_recta(0, 0, get_pendiente(0, 0, 5, 5)), "y = x")
168         self.assertEqual(get_ecuacion_recta(0, 3, get_pendiente(0, 3, 3, 0)), "y = -x + 3")
169         self.assertEqual(get_ecuacion_recta(0, -3, get_pendiente(0, -3, 1, -1)), "y = 2x - 3")
```

test_trayectorias_casos_especiales revisa los puntos que genera el DDA: en la vertical hacia abajo todos tienen $X = 3$ y termina en $(3, 1)$; en la horizontal hacia la izquierda las X van 6, 5, 4, 3, 2, 1; la línea al revés empieza en A y termina en B; y dos puntos a menos de una unidad producen 2 puntos. `all(...)` devuelve `True` solo si la condición se cumple para todos los elementos.

test_ecuaciones_casos comprueba los textos simplificados: « $y = 2$ », « $y = x$ », « $y = -x + 3$ » y « $y = 2x - 3$ ».

test_app.py · líneas 171-213

```
171     def test_interfaz_todos_los_casos(self):
172         app = ctk.CTk()
173         app.withdraw()
174         container = ctk.CTkFrame(app)
175         container.pack()
176         page = create_puntos_colineales_page(container)
177         page.pack()
178
179         entries = []
180         calc_btn = None
181
182         def search(widget):
183             nonlocal calc_btn
184             if isinstance(widget, ctk.CTkEntry):
185                 entries.append(widget)
186             elif isinstance(widget, ctk.CTkButton) and "Calcular" in widget.cget("text"):
187                 calc_btn = widget
188             for child in widget.winfo_children():
189                 search(child)
```

```

190
191     search(page)
192     self.assertEqual(len(entries), 4)
193
194     for (x1, y1), (x2, y2), esperado in self.CASOS:
195         with self.subTest(caso=esperado):
196             for entry, valor in zip(entries, (x1, y1, x2, y2)):
197                 entry.delete(0, "end")
198                 entry.insert(0, str(valor))
199             calc_btn.invoke()
200             self.assertTrue(current_data.get("has_data"))
201             self.assertEqual(current_data.get("caso"), esperado)
202
203     # Casos sin pendiente definida
204     self.assertEqual(current_data.get("caso"), "Dibujada al revés (de derecha a
izquierda)")
205     entries[0].delete(0, "end"); entries[0].insert(0, "3")
206     entries[1].delete(0, "end"); entries[1].insert(0, "1")
207     entries[2].delete(0, "end"); entries[2].insert(0, "3")
208     entries[3].delete(0, "end"); entries[3].insert(0, "7")
209     calc_btn.invoke()
210     self.assertIsNone(current_data.get("pendiente"))
211     self.assertEqual(current_data.get("equation"), "x = 3")
212
213     app.destroy()

```

test_interfaz_todos_los_casos usa la interfaz real: busca las 4 cajas de texto y el botón «Calcular» (igual que la prueba anterior), y para cada caso de la lista escribe las coordenadas, presiona el botón y comprueba que `current_data["caso"]` sea el esperado. `zip(entries, (x1, y1, x2, y2))` empareja cada caja con su valor. Al final prueba una vertical y verifica que la pendiente sea `None` y la ecuación « $x = 3$ ».

Las líneas 216-217 permiten ejecutar las pruebas con `python test_app.py`.

12. requirements.txt y cómo ejecutar

`requirements.txt` lista las bibliotecas externas con su versión exacta (`nombre==versión`). `pip install -r requirements.txt` las instala todas. Las que el código usa directamente son `customtkinter`, `CTkTable`, `matplotlib` y `pillow`; las demás (`numpy`, `contourpy`, `darkdetect`, etc.) son dependencias que esas bibliotecas necesitan a su vez.

OBSERVACIÓN

El archivo también incluye `flet`, `flet-desktop` y otras bibliotecas relacionadas (`httpx`, `oauthlib`, `rich`...). Ningún archivo del proyecto importa `flet`; probablemente quedó de una versión anterior. No afecta el funcionamiento, solo agrega instalaciones innecesarias.

Entorno virtual

Un **entorno virtual** (carpeta `.venv`) es una instalación de Python aislada para este proyecto, para que sus bibliotecas no choquen con las de otros proyectos. Está excluida de Git mediante `.gitignore` (junto con `__pycache__`, la carpeta donde Python guarda versiones precompiladas de los archivos).

Paso	Windows (PowerShell)	Linux / macOS
Crear el entorno (una vez)	<code>python -m venv .venv</code>	<code>python3 -m venv .venv</code>
Activarlo	<code>.\.venv\Scripts\Activate.ps1</code>	<code>source .venv/bin/activate</code>
Instalar bibliotecas (una vez)	<code>pip install -r requirements.txt</code>	
Ejecutar la app	<code>python main.py</code>	
Ejecutar las pruebas	<code>python -m unittest test_app.py</code>	

13. Recorrido completo: de «python main.py» a la recta dibujada

Al arrancar

- 1 Python empieza a leer `main.py`. En la línea 7 importa `pages/index.py`, que importa `Futuro.py` y `puntos_colineales.py`; estos importan `icons.py`. Se crean todas las funciones y las variables globales vacías (`current_data = {}`, `_pages = {}`...). Aún no hay ventana.
- 2 Se activa el modo oscuro y se crea la ventana `root` de 1340×840 .
- 3 Se construye la barra lateral: logotipo, subtítulo, separador, botones de navegación. Para cada icono, `get_icon` lo dibuja a 4x con Pillow, lo reduce y lo guarda en caché.
- 4 `register_nav_listener(on_page_changed)` suscribe el resaltado de botones.
- 5 Se crea el interruptor de tema, encendido.
- 6 Se crea el contenedor y se llama a `init_pages`, que ejecuta `create_puntos_colineales_page`: construye encabezado, formulario, tarjetas, tabla vacía y plano vacío; suscribe `update_plot`; escribe 1, 2, 5, 8 en las cajas. Luego ejecuta `future_page`.
- 7 Ambas páginas se apilan en la misma celda y `show_page("puntos_colineales")` trae al frente la principal y resalta su botón.
- 8 `root.mainloop()`: aparece la ventana y el programa queda esperando eventos.

Al presionar «Calcular Trayectoria»

- 1 El ciclo de eventos detecta el clic y ejecuta la `lambda` del botón, que llama a `calculate()`.
- 2 Lee las cajas: "1", "2", "5", "8" $\rightarrow$ 1, 2, 5, 8. Son números válidos.
- 3 Calcula $\Delta X = 4$, $\Delta Y = 6$ y los sentidos, y clasifica el trazo: $|6| > |4|$ y ambos positivos $\rightarrow$ «Parada, subiendo hacia la derecha». Calcula `limit = 10`.
- 4 Como $\Delta X \neq 0$, sigue el camino normal: $m = 1.5$, $b = 0.5$, ángulo 56.3° , ecuación « $y = 1.5x + 0.5$ », extremos $(-10, -14.5)$ y $(10, 15.5)$. Luego genera los 7 puntos de trayectoria.
- 5 Guarda todo en `current_data`.
- 6 Actualiza las 4 tarjetas y la franja del caso, reconstruye la tabla con 7 filas y el contador.
- 7 `render_plot` borra el plano, lo redibuja con los colores del tema y pinta la recta, los 7 puntos, A, B, sus etiquetas y la leyenda.
- 8 El programa vuelve a esperar eventos.

Otros eventos

- **Datos inválidos** (por ejemplo, «abc»): `float()` falla → ventana «Dato Inválido» → no cambia nada más.
- **$X_1 = X_2$ (vertical)**: se calcula por el camino especial: pendiente «Indefinida», ángulo 90° , ecuación « $x = X_1$ », y se dibuja una línea vertical con su trayectoria.
- **$A = B$ (un solo punto)**: no hay recta; las tarjetas muestran «Indefinida» y «Punto (x, y)», la tabla tiene 1 fila y en el plano aparece un solo marcador « $A = B$ ».
- **Limpiar**: `clear()` vacía cajas, tarjetas, tabla y `current_data`, y dibuja el plano vacío.
- **Mover el interruptor**: `toggle_theme()` cambia el modo; CustomTkinter recolorea sus widgets y cambia los iconos a su versión clara u oscura; `notify_theme_to_plot()` → `update_plot()` → `render_plot()` vuelve a dibujar la gráfica con los mismos datos de `current_data`, ahora con los colores del nuevo tema.
- **Módulo Futuro**: `show_page("futuro")` trae esa página al frente y resalta su botón. La página de puntos queda intacta detrás.
- **Cerrar la ventana**: termina `mainloop()` y el programa finaliza.

14. Glosario

Término	Significado
Aserción	Comprobación dentro de una prueba: si no se cumple, la prueba falla.
Caché	Lugar donde se guardan resultados ya calculados para no repetir el trabajo.
Caso de trazo	Tipo de recta según ΔX y ΔY (horizontal, vertical, diagonal, acostada, parada...). La app reconoce 11.
Callback	Función que se entrega a otra parte del programa para que la llame después, cuando ocurra algo.
Ciclo de eventos	Bucle (<code>mainloop</code>) que espera acciones del usuario y ejecuta la función correspondiente.
Cierre (closure)	Función interna que recuerda y usa variables de la función que la contiene.
DDA	Algoritmo que genera puntos intermedios de una recta avanzando en pasos iguales.
Diccionario	Colección de pares clave $\rightarrow$ valor.
Excepción	Error durante la ejecución; puede atraparse con <code>try/except</code> .
Gestor de geometría	<code>pack</code> , <code>grid</code> o <code>place</code> : deciden dónde y de qué tamaño se muestra un widget.
Hexadecimal	Sistema de numeración base 16 (0-9 y A-F), usado para escribir colores.
Lambda	Función corta sin nombre, escrita en una línea.
Módulo / paquete	Un archivo <code>.py</code> / una carpeta de archivos <code>.py</code> .
Ordenada al origen	Valor b donde la recta cruza el eje Y.
Patrón observador	Forma de organizar el código en la que los interesados se suscriben a una lista y se les avisa cuando algo cambia.
Pendiente	Inclinación de la recta: cambio en Y por cada unidad de cambio en X. En una recta vertical es indefinida.
Rasterizar	Convertir una figura continua (una recta) en puntos discretos (píxeles).
Recta acostada / parada	Recta con inclinación menor / mayor de 45° .
Recursión	Cuando una función se llama a sí misma (por ejemplo, para recorrer un árbol de widgets o clasificar una línea al revés).
RGBA	Color como rojo, verde, azul y opacidad, cada uno de 0 a 255.
Sobremuestreo	Dibujar a mayor tamaño y reducir, para suavizar bordes.
Tupla	Secuencia ordenada que no se puede modificar.
Valor absoluto	El número sin signo: $ -4 = 4$.
Widget	Cualquier elemento visible de la interfaz: botón, etiqueta, caja de texto, marco...